Università degli Studi di Firenze
Scuola di Ingegneria
Corso di Laurea in Ingegneria Informatica

DriveHub

Sistema informativo per la vendita e il noleggio di veicoli

Candidati:

Federico D'Amodio
Matteo Orlandi

Corso:
Ingegneria del Software

Docente:
Prof. Enrico Vicario

Anno Accademico 2025/2026

Indice

1 Introduzione **1**
- 1.1 Descrizione . . . 1
- 1.2 Stack tecnologico . . . 1
- 1.3 Semplificazioni adottate . . . 2

2 Analisi dei requisiti e progettazione **3**
- 2.1 Diagramma dei casi d'uso . . . 3
- 2.2 Template dei casi principali . . . 4
  - 2.2.1 Prenotare un test drive . . . 4
  - 2.2.2 Noleggiare un veicolo . . . 5
  - 2.2.3 Richiedere e completare un acquisto . . . 7
  - 2.2.4 Proporre un veicolo al concessionario . . . 8
  - 2.2.5 Autorizzare una richiesta di acquisto . . . 10
  - 2.2.6 Gestire lo sconto su un veicolo . . . 11
- 2.3 Mockups . . . 13
  - 2.3.1 Autenticazione . . . 13
  - 2.3.2 Area Customer . . . 14
  - 2.3.3 Area Salesman . . . 17
  - 2.3.4 Area Manager . . . 20
- 2.4 Page Navigation Diagram . . . 22
- 2.5 Package Diagram . . . 23
- 2.6 Class Diagram . . . 24
  - 2.6.1 Domain Model . . . 24
  - 2.6.2 Business Layer . . . 25
  - 2.6.3 DAO . . . 26
  - 2.6.4 Presentation Layer . . . 27
- 2.7 Sequence Diagram . . . 27
  - 2.7.1 Acquisto di un veicolo . . . 28
  - 2.7.2 Prenotazione di un noleggio . . . 29
  - 2.7.3 Acquisizione di un veicolo del Customer . . . 30
- 2.8 Database . . . 31
  - 2.8.1 Schema ER . . . 31
  - 2.8.2 Modello relazionale . . . 32
  - 2.8.3 Utenti, vincoli e semplificazioni . . . 33

3 Implementazione **35**
- 3.1 Domain Model . . . 35
  - 3.1.1 Users . . . 35

3.1.2 Vehicles . . . 36
3.1.3 Rentals . . . 36
3.1.4 Sales . . . 37
3.1.5 Observer . . . 38
3.2 Business Layer . . . 38
3.2.1 Core: unità di lavoro e transazioni . . . 38
3.2.2 Services: responsabilità . . . 39
3.2.3 Services: flussi applicativi . . . 39
3.2.4 Strategies: prezzi e sconti . . . 42
3.2.5 Security e Banking: autenticazione e conto . . . 43
3.2.6 Observer e gestione dell'inventario . . . 43
3.3 Presentation Layer . . . 44
3.3.1 Controller e aree applicative . . . 44
3.3.2 Creazione dei controller e navigazione . . . 44
3.3.3 Gateway, dati della UI e aggiornamento . . . 45
3.4 Persistenza: livello DAO e mapping oggetti/tabelle . . . 45
3.4.1 Porte DAO e implementazioni JDBC . . . 46
3.4.2 Mapping degli utenti e vista unificata . . . 46
3.4.3 Mapping delle operazioni e valori opzionali . . . 47
3.4.4 Vincoli SQL e controlli dei servizi . . . 47
3.4.5 Connessione condivisa e Unit of Work . . . 47
3.5 Idiomi e Design Pattern . . . 48
3.5.1 Observer . . . 48
3.5.2 Strategy . . . 49
3.5.3 Metodi statici di creazione . . . 49
3.5.4 Istanza centrale per la navigazione . . . 49
3.5.5 DAO e Unit of Work . . . 50
3.5.6 Idiomi Java usati nei confini tra livelli . . . 50

4 Testing **51**
4.1 Strategia di verifica . . . 51
4.1.1 Tecnologie e strumenti . . . 51
4.2 Livello 1: Business Logic . . . 52
4.2.1 Invarianti del dominio . . . 52
4.2.2 AuthServiceTest . . . 52
4.2.3 PricingAndSecurityTest . . . 52
4.2.4 SalesServiceTest . . . 52
4.2.5 RentalServiceTest e BankAccountServiceTest . . . 53
4.2.6 TransactionAndObserverTest . . . 53
4.3 Livello 2: Persistence Layer (DAO) . . . 54
4.3.1 PostgresDaoIntegrationTest . . . 54
4.3.2 DatabaseBootstrapIntegrationTest . . . 54
4.4 Verifiche funzionali e della presentazione . . . 54
4.5 Esecuzione della suite e portata dei risultati . . . 55

Capitolo 1

Introduzione

1.1 Descrizione

DriveHub è un'applicazione desktop per le attività essenziali di un concessionario: presentare i veicoli disponibili, organizzare test drive e noleggi, gestire le richieste di acquisto e valutare le auto proposte dai clienti. Un unico catalogo collega queste operazioni allo stato di ciascun veicolo, così che le scelte degli utenti si riflettano sulla sua disponibilità.

Il sistema prevede tre figure operative:

- **Cliente** (`CLIENT`)**:** si registra, consulta il catalogo, prenota un test drive o un noleggio, versa l'acconto per richiedere un acquisto e propone al concessionario un proprio veicolo. Può consultare le proprie operazioni e usare un conto dimostrativo per i movimenti previsti dall'applicazione.
- **Venditore** (`SALESMAN`)**:** esamina i veicoli proposti dai clienti e decide se accettarli o rifiutarli; autorizza il completamento delle richieste di acquisto e può cambiare la destinazione di un veicolo tra vendita e noleggio. Nella propria area consulta anche richieste, noleggi e test drive.
- **Manager** (`MANAGER`)**:** cura il catalogo inserendo marche, modelli e veicoli, applica sconti a modelli o singole auto e consulta le vendite registrate.

Le tre aree condividono le informazioni sul veicolo, ma espongono operazioni diverse in base al ruolo.

1.2 Stack tecnologico

- **Linguaggio e interfaccia:** Java 21 e JavaFX 21.0.2. I layout sono definiti in FXML, lo stile in CSS e le azioni delle schermate nei controller.
- **Persistenza:** PostgreSQL tramite driver JDBC. All'avvio, se la connessione non riesce, l'applicazione usa un database H2 in memoria in modalità compatibile con PostgreSQL, inizializzato con dati dimostrativi.
- **Build e verifica:** Maven gestisce dipendenze e compilazione; JUnit 5 e Mockito supportano i test, JaCoCo la misurazione della copertura.

1.3 Semplificazioni adottate

Per mantenere il progetto realizzabile e dimostrabile, alcuni processi del concessionario sono rappresentati in forma ridotta:

- **Pagamenti interni:** acconti, saldi, noleggi e rimborsi modificano il saldo di un `BankAccount` locale. Non sono integrati banche o circuiti di pagamento.
- **Account dello staff:** la registrazione consente di scegliere anche i ruoli Venditore e Manager. È una scelta del prototipo; l'attribuzione di questi ruoli in un sistema operativo richiederebbe un controllo amministrativo.
- **Modalità dimostrativa:** se l'avvio con PostgreSQL non riesce, l'applicazione crea automaticamente un database H2 nella memoria del processo e lo popola con utenti e veicoli di esempio. È così possibile provare acquisti, noleggi e test drive senza configurare un server esterno. Gli account creati e le operazioni effettuate restano disponibili durante quella sessione, ma vengono persi quando si chiude l'applicazione; al successivo avvio in modalità dimostrativa si riparte dai dati di esempio. Con PostgreSQL, invece, i dati sono salvati su disco e restano disponibili agli avvii successivi.
- **Processi aziendali:** il sistema registra le operazioni sul veicolo senza modellare sedi, fornitori, officina, consegna fisica, contratti, fatturazione o assistenza successiva alla vendita.

Capitolo 2

Analisi dei requisiti e progettazione

2.1 Diagramma dei casi d'uso

Il diagramma mostra le funzioni offerte ai tre ruoli di DriveHub. L'attore generale *User* raccoglie registrazione e accesso; *Customer*, *Salesman* e *Manager* ereditano queste possibilità e accedono alle operazioni delle rispettive aree. Il Customer consulta il catalogo e avvia le operazioni commerciali; il Salesman valuta le proposte e segue le pratiche; il Manager mantiene catalogo e sconti e consulta le vendite. I sei casi d'uso dettagliati sotto comprendono quattro operazioni del Customer, l'autorizzazione di un acquisto da parte del Salesman e la gestione di uno sconto da parte del Manager.

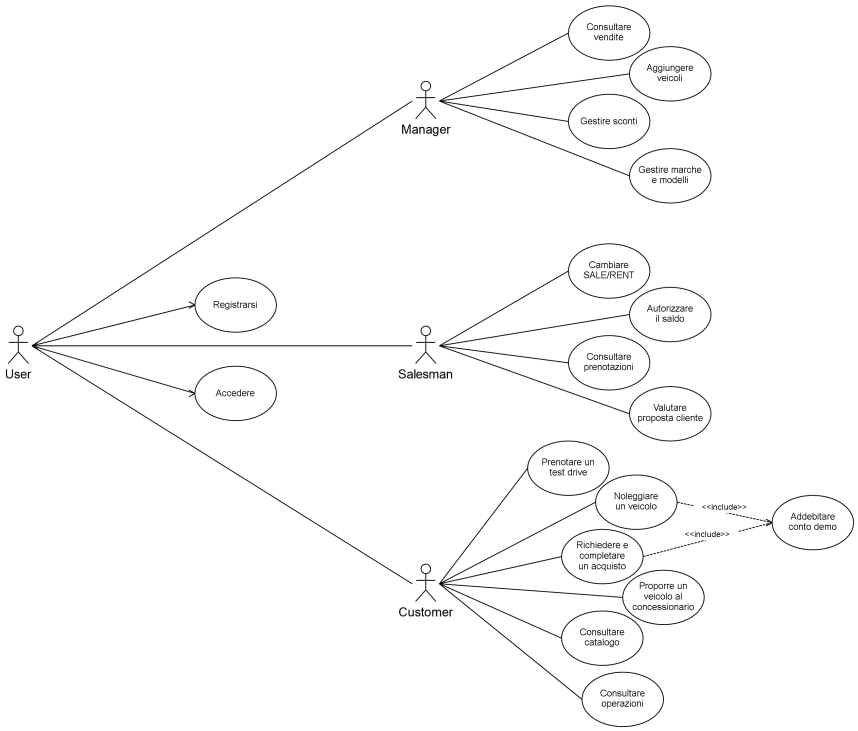


Figura 2.1: Casi d'uso di DriveHub nel codice corrente. Diagramma sorgente: `docs/Users SWE.drawio`.

2.2 Template dei casi principali

Ogni caso d'uso specifica nome, livello, descrizione, pre-condizioni, post-condizioni, attori, test e flussi. Il flusso principale è numerato a partire da 1 in ciascun template. Le alternative sono identificate da A1, A2 e così via, con il rimando al passo principale da cui si diramano. I riferimenti ai test distinguono i comportamenti coperti dalle alternative prive di una prova specifica. Le anomalie implementative sono richiamate separatamente dai flussi quando necessario.

2.2.1 Prenotare un test drive

Tabella 2.1: Caso d'uso: Prenotare un test drive

Campo	Contenuto
Nome	Prenotare un test drive
Livello	Obiettivo Utente
Descrizione	Il Customer prenota una prova di guida di un'ora per un veicolo disponibile nel catalogo.
Pre-condizioni	Il Customer ha effettuato l'accesso; il veicolo esiste ed è in stato `SALE` o `RENT`; la data e la fascia oraria scelte non sono passate.
Post-condizioni	Una prenotazione di test drive è registrata per il Customer, il veicolo e lo slot selezionato.
Attori	Customer
Test	`UserGoalsFunctionalTest.bookTestDrive` esercita il percorso principale su H2; `RentalWorkflowTest` verifica le regole dell'entità. Non è presente una prova funzionale specifica per ciascuna alternativa o per richieste concorrenti.
Flusso Principale	1. Il Customer seleziona un veicolo dal catalogo e apre la scheda dei test drive. 2. Sceglie una data e uno degli orari proposti, dalle 09:00 alle 16:00. 3. Il sistema verifica che il veicolo sia idoneo, che il Customer abbia meno di tre test drive futuri e che il veicolo non sia noleggiato nella data scelta. 4. Il sistema verifica che lo slot di un'ora non sia già occupato per quel veicolo. 5. Il sistema salva la prenotazione e ne comunica l'esito al Customer.

Continua nella pagina successiva...

Tabella 2.1 – Continua dalla pagina precedente

Campo	Contenuto
Flussi Alternativi	**A1 (passo 2). Data o orario non validi:** se lo slot è nel passato o fuori dalla fascia consentita, il sistema rifiuta la prenotazione. **A2 (passo 3). Veicolo non idoneo o limite raggiunto:** un veicolo non in vendita né a noleggio, oppure tre test drive futuri già registrati, impedisce il salvataggio. **A3 (passi 3–4). Veicolo indisponibile:** un noleggio nel giorno scelto o un altro test drive nello stesso slot produce un conflitto. **A4 (passo 4). Richieste concorrenti:** il controllo dello slot precede l'inserimento; il database non impone un vincolo univoco sullo slot, quindi due richieste simultanee potrebbero entrambe riuscire. Inoltre il servizio riconduce i minuti forniti all'inizio dell'ora: un input diretto come 10:30 è trattato come 10:00.

2.2.2 Noleggiare un veicolo

Tabella 2.2: Caso d'uso: Noleggiare un veicolo

Campo	Contenuto
Nome	Noleggiare un veicolo
Livello	Obiettivo Utente
Descrizione	Il Customer prenota un veicolo per un intervallo di giorni e paga anticipatamente l'intero importo con il conto dimostrativo.
Pre-condizioni	Il Customer ha effettuato l'accesso; il veicolo esiste ed è in stato `RENT`; la data iniziale non è passata e la data finale non precede quella iniziale.
Post-condizioni	Il noleggio è registrato, l'importo è sottratto dal saldo del Customer e i test drive sul veicolo nei giorni prenotati sono cancellati. Le scritture nel database avvengono nella stessa transazione.
Attori	Customer

Continua nella pagina successiva...

Tabella 2.2 – Continua dalla pagina precedente

Campo	Contenuto
Test	`RentalServiceTest.bookRentalDebitsAccount` verifica il servizio con DAO simulati; `CheckoutFunctionalTest.rentalBookingWorkflow` verifica il percorso principale su H2. Le alternative di conflitto e la concorrenza non hanno un test funzionale dedicato nella suite corrente.
Flusso Principale	1. Il Customer seleziona un veicolo disponibile per il noleggio e indica il primo e l'ultimo giorno. 2. Il sistema mostra un preventivo. Nel conteggio sono inclusi sia il giorno iniziale sia quello finale. 3. Il Customer conferma la prenotazione. 4. Il sistema verifica che non vi siano noleggi sovrapposti e calcola l'importo: tariffa giornaliera pari allo 0,1% del prezzo del veicolo, con un fattore 0,95 applicato per ogni giorno aggiuntivo. 5. Il sistema addebita l'intero importo sul conto dimostrativo e registra il noleggio. 6. Il sistema cancella gli eventuali test drive sul veicolo nei giorni prenotati e conferma l'operazione.
Flussi Alternativi	**A1 (passo 1). Veicolo o date non validi:** il sistema rifiuta la richiesta senza registrare il noleggio. **A2 (passo 4). Periodo occupato:** se esiste un noleggio che condivide almeno un giorno, l'operazione è rifiutata. **A3 (passo 4). Richieste concorrenti:** la verifica delle sovrapposizioni non è protetta da un vincolo di esclusione nel database; due richieste simultanee potrebbero entrambe riuscire. **A4 (passo 5). Saldo insufficiente:** l'addebito fallisce; la transazione annulla anche le altre scritture.

2.2.3 Richiedere e completare un acquisto

Tabella 2.3: Caso d'uso: Richiedere e completare un acquisto

Campo	Contenuto
Nome	Richiedere e completare un acquisto
Livello	Obiettivo Utente
Descrizione	Il Customer riserva un veicolo in vendita versando un acconto; dopo l'autorizzazione del Salesman, paga il saldo e conclude l'acquisto. Le azioni possono avvenire in sessioni diverse.
Pre-condizioni	Customer e Salesman hanno effettuato l'accesso alle rispettive aree; il veicolo esiste ed è in stato `SALE`; il Customer dispone dell'importo necessario per l'acconto.
Post-condizioni	Dopo l'acconto il veicolo è `BLOCKED` e la richiesta è registrata. Dopo il saldo il veicolo è `SOLD`, la richiesta è eliminata, la vendita è registrata e gli eventuali test drive futuri sono cancellati. Ciascuna fase ha una propria transazione.
Attori	Customer (primario), Salesman
Test	`CheckoutFunctionalTest.completePurchaseWorkflow` verifica il percorso principale su H2; `cancelPurchaseWorkflowWithPenalty` verifica l'annullamento. `SalesServiceTest` verifica avvio e penale con mock. Autorizzazione assente, scadenza e variazione del prezzo non sono coperti da una prova funzionale dedicata.
Flusso Principale	1. Il Customer seleziona un veicolo in vendita; l'interfaccia calcola l'acconto, pari al 20% del prezzo visualizzato. 2. Il Customer conferma: il sistema addebita l'acconto, registra la richiesta e blocca il veicolo. 3. Il Salesman consulta la richiesta e la autorizza; il sistema registra il venditore e fissa la scadenza a un mese. 4. Il Customer seleziona la richiesta autorizzata e conferma il pagamento del saldo. 5. Il sistema calcola la differenza tra il prezzo corrente del veicolo e l'acconto già versato, quindi addebita l'eventuale residuo. 6. Il sistema registra la vendita, elimina la richiesta, pone il veicolo in stato `SOLD` e cancella i test drive futuri relativi al veicolo.

Continua nella pagina successiva...

Tabella 2.3 – Continua dalla pagina precedente

Campo	Contenuto
Flussi Alternativi	**A1 (passo 2). Veicolo non disponibile o saldo insufficiente:** la richiesta non viene creata e le scritture della fase sono annullate. **A2 (passo 4). Autorizzazione assente:** l'interfaccia impedisce di avviare il saldo. Il servizio di completamento non verifica autonomamente autorizzazione e scadenza: una chiamata diretta potrebbe aggirare quel controllo. **A3 (passo 4). Annullamento o scadenza:** la richiesta viene eliminata e il veicolo torna in vendita; il sistema trattiene il 20% dell'acconto e rimborsa l'80%. Le richieste scadute sono elaborate all'avvio e durante alcune consultazioni, senza un timer dedicato. **A4 (passo 5). Prezzo variato:** la richiesta non conserva un prezzo finale bloccato; il residuo è calcolato sul prezzo del veicolo letto al momento del saldo.

Anomalia del prezzo al saldo. Il passo 5 descrive il comportamento attuale del servizio: il residuo è calcolato da `Vehicle.price`, mentre l'acconto deriva dal prezzo di catalogo calcolato da `quotePrice`. Il totale pagato può quindi differire dal prezzo mostrato al Customer. Si tratta di un'incoerenza implementativa, approfondita nella sezione 3.2.4.

2.2.4 Proporre un veicolo al concessionario

Tabella 2.4: Caso d'uso: Proporre un veicolo al concessionario

Campo	Contenuto
Nome	Proporre un veicolo al concessionario
Livello	Obiettivo Utente
Descrizione	Il Customer propone alla concessionaria il proprio veicolo. Il sistema calcola una valutazione; un Salesman può accettare la proposta e accreditare l'importo sul conto dimostrativo del Customer.
Pre-condizioni	Il Customer ha effettuato l'accesso; sceglie marca e modello, inserisce VIN, chilometri e anno di immatricolazione validi. Se manca la targa, i chilometri devono essere zero.

Continua nella pagina successiva...

Tabella 2.4 – Continua dalla pagina precedente

Campo	Contenuto
Post-condizioni	Dopo l'invio, il veicolo è in stato `WAITING` e la proposta è registrata. Se accettata, l'importo è accreditato al Customer, la proposta è eliminata, la spesa della concessionaria è registrata e il veicolo passa a `SALE`. Invio e decisione sono due transazioni distinte.
Attori	Customer (primario), Salesman
Test	`CheckoutFunctionalTest.customerProposeAndApprove` esercita proposta e approvazione su H2. Rifiuto, riuso di VIN `SOLD` e accredito fallito non hanno un test funzionale dedicato nella suite corrente.
Flusso Principale	1. Il Customer apre la schermata di vendita del proprio veicolo e seleziona marca e modello. 2. Inserisce VIN, eventuale targa, chilometri e anno di immatricolazione. 3. Il sistema mostra una valutazione calcolata dal prezzo di listino del modello, tenendo conto di targa, età e chilometri. 4. Il Customer conferma la proposta; il sistema registra il veicolo in stato `WAITING` e la richiesta con l'importo stimato. 5. Il Salesman esamina i veicoli in attesa e approva la proposta. 6. Il sistema accredita la valutazione al Customer, registra l'operazione con importo negativo per la concessionaria, elimina la richiesta e mette il veicolo in vendita.
Flussi Alternativi	**A1 (passo 2). Dati non validi:** l'interfaccia rifiuta VIN mancante o troppo lungo, chilometri negativi, anno fuori intervallo e veicolo senza targa con chilometri diversi da zero. **A2 (passo 4). VIN già attivo:** se il VIN appartiene a un veicolo non `SOLD`, la proposta è rifiutata. Se il VIN appartiene a un veicolo `SOLD`, il sistema riutilizza quella scheda e la pone in stato `WAITING`. **A3 (passo 5). Proposta rifiutata:** il Salesman elimina la richiesta; il veicolo torna `SOLD` se ha operazioni precedenti, altrimenti viene eliminato dal catalogo. **A4 (passo 6). Accredito non riuscito:** le modifiche della decisione vengono annullate dalla transazione.

2.2.5 Autorizzare una richiesta di acquisto

Tabella 2.5: Caso d'uso: Autorizzare una richiesta di acquisto

Campo	Contenuto
Nome	Autorizzare una richiesta di acquisto
Livello	Obiettivo Utente
Descrizione	Il Salesman autorizza il Customer a completare un acquisto per cui è già stato versato un acconto. L'autorizzazione assegna il venditore alla pratica e stabilisce il termine per il saldo.
Pre-condizioni	Il Salesman ha effettuato l'accesso alla propria area; esiste una richiesta di acquisto con acconto positivo e il relativo veicolo è stato riservato dal Customer.
Post-condizioni	La richiesta conserva l'identificativo del Salesman e una scadenza pari alla data di autorizzazione più un mese. Il veicolo resta `BLOCKED`; in questa fase non avvengono addebiti.
Attori	Salesman (primario), Customer (beneficiario dell'autorizzazione)
Test	`CheckoutFunctionalTest.completePurchaseWorkflow` verifica che l'autorizzazione imposti la scadenza a un mese e che il successivo acquisto registri il venditore. La selezione nella GUI e i rami di errore non sono verificati da quel test.
Flusso Principale	1. Il Salesman consulta le richieste di acquisto e seleziona la pratica da autorizzare. 2. Attiva il comando di autorizzazione. 3. Il sistema recupera la richiesta e verifica che si tratti di un acquisto, riconoscibile dall'acconto positivo. 4. Il sistema salva nella richiesta l'identificativo del Salesman e la data di scadenza, in un'unica transazione. 5. L'interfaccia conferma l'esito e aggiorna l'elenco. Il Customer può procedere al saldo dalla propria area.
Flussi Alternativi	**A1 (passo 1). Nessuna richiesta selezionata:** l'interfaccia mostra un messaggio e non invia l'autorizzazione. **A2 (passo 3). Richiesta assente o di tipo diverso:** una richiesta non più presente produce un errore; una proposta di cessione è rifiutata dall'interfaccia e dal servizio di autorizzazione. Nessuna autorizzazione viene salvata. **A3 (passo 4). Salvataggio non riuscito:** la transazione viene annullata e l'interfaccia comunica l'errore.

2.2.6 Gestire lo sconto su un veicolo

Tabella 2.6: Caso d'uso: Gestire lo sconto su un veicolo

Campo	Contenuto
Nome	Gestire lo sconto su un veicolo
Livello	Obiettivo Utente
Descrizione	Il Manager applica, modifica o rimuove una promozione sul singolo veicolo. Il catalogo del Customer considera questa percentuale nel calcolo del prezzo esposto.
Pre-condizioni	Il Manager ha effettuato l'accesso alla propria area e il veicolo è presente nel catalogo.
Post-condizioni	La percentuale di sconto del veicolo è salvata nel database. Il successivo calcolo del preventivo considera il maggiore fra sconto del veicolo e sconto del modello, senza sommarli.
Attori	Manager
Test	`UserGoalsFunctionalTest.managerVehicleDiscount` applica il 10%, verifica la riduzione del preventivo e controlla il ritorno al prezzo precedente dopo la rimozione. Il test attraversa servizio e persistenza H2; non esercita i comandi della GUI né il saldo dell'acquisto.
Flusso Principale	1. Il Manager consulta il parco veicoli e seleziona un esemplare. 2. Inserisce una percentuale intera compresa fra 0 e 50 e attiva il comando per applicarla. 3. Il sistema valida la percentuale e salva lo sconto del veicolo, sostituendo quello eventualmente già presente nella stessa transazione. 4. L'interfaccia conferma l'esito e aggiorna i dati. Alla successiva consultazione del catalogo, il preventivo viene ricalcolato considerando la promozione salvata.

Continua nella pagina successiva...

Tabella 2.6 – Continua dalla pagina precedente

Campo	Contenuto
Flussi Alternativi	**A1 (passo 1). Nessun veicolo selezionato:** l'interfaccia richiede una selezione e non invia modifiche. **A2 (passo 2). Percentuale non valida:** campo vuoto, valore non intero o fuori dall'intervallo 0–50 causano un errore; lo sconto precedente resta invariato. **A3 (passo 2). Rimozione dello sconto:** il Manager sceglie il comando di rimozione; il sistema elimina la promozione individuale. L'eventuale sconto del modello resta applicabile al preventivo. **A4 (passo 3). Salvataggio non riuscito:** la transazione viene annullata e l'interfaccia comunica l'errore.

Limite del flusso di acquisto. Lo sconto interviene sul preventivo di catalogo; il saldo usa invece il prezzo memorizzato nel veicolo, secondo l'anomalia descritta nella sezione 3.2.4.

2.3 Mockups

Le schermate seguenti sono state acquisite dall'applicazione JavaFX corrente, avviata con il database dimostrativo in memoria. I dati visualizzati appartengono al popolamento di prova. Le viste sono presentate nell'ordine in cui l'utente le incontra e poi raggruppate per ruolo.

2.3.1 Autenticazione

Schermata iniziale

La pagina di benvenuto è il punto di ingresso dell'applicazione e offre i collegamenti all'accesso e alla creazione di un account.



Figura 2.2: Schermata iniziale di DriveHub.

Accesso

La schermata di login richiede email e password. Dopo la verifica delle credenziali, l'utente viene indirizzato all'area corrispondente al proprio ruolo.

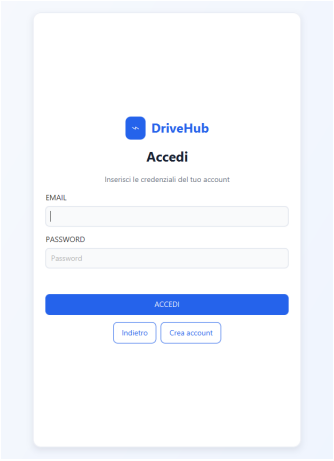


Figura 2.3: Schermata di accesso.

Registrazione

La registrazione raccoglie nome, cognome, email, password e ruolo. Per il Customer viene creato automaticamente un conto dimostrativo. La scelta pubblica dei ruoli Salesman e Manager è una semplificazione del prototipo.

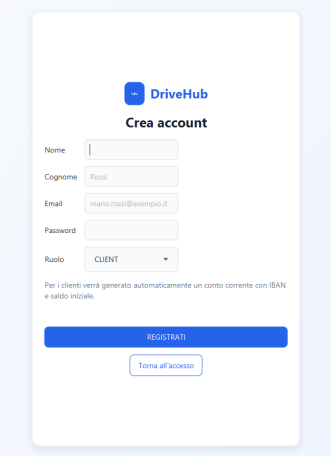


Figura 2.4: Schermata di registrazione.

2.3.2 Area Customer

Catalogo veicoli

Il catalogo presenta i veicoli con targa, VIN, modello, anno, chilometri, prezzo e stato. Il Customer può cercare per testo, filtrare per modalità e scegliere un veicolo per avviare acquisto, noleggio o test drive.

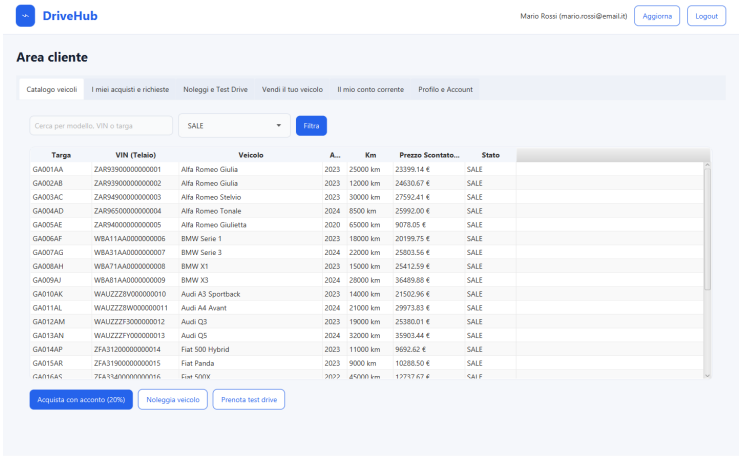


Figura 2.5: Catalogo dei veicoli nell'area Customer.

Acquisti e richieste

La scheda distingue le richieste ancora aperte dallo storico delle compravendite. Per una richiesta di acquisto autorizzata il Customer può pagare il saldo; può anche annullare l'acquisto con rimborso parziale dell'acconto.

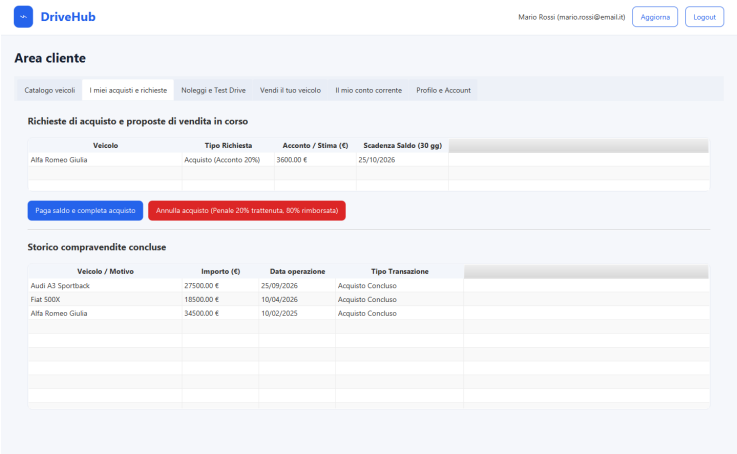


Figura 2.6: Richieste di acquisto e storico del Customer.

Noleggi e test drive

Le due operazioni condividono una scheda: il noleggio richiede un intervallo di date, mentre il test drive richiede una data e uno slot di un'ora fra le 09:00 e le 16:00. Le tabelle sottostanti mostrano le prenotazioni del Customer.

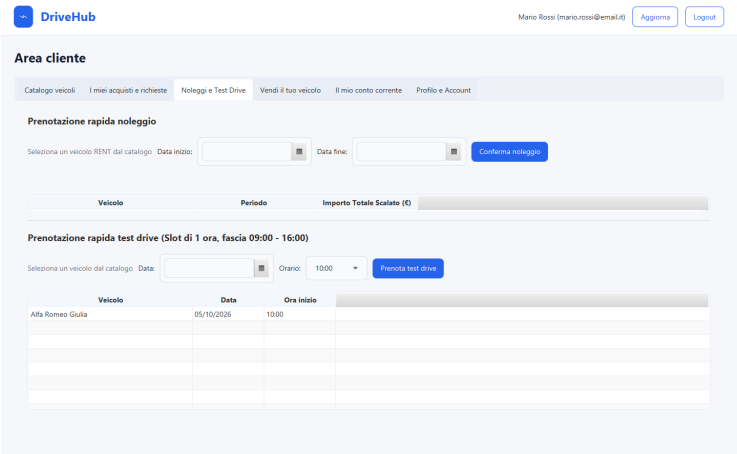


Figura 2.7: Prenotazione di noleggi e test drive.

Proposta di vendita del proprio veicolo

Il Customer seleziona marca e modello e inserisce VIN, targa, chilometri e anno. Il sistema calcola una valutazione prima dell'invio della proposta, che sarà esaminata dal Salesman.

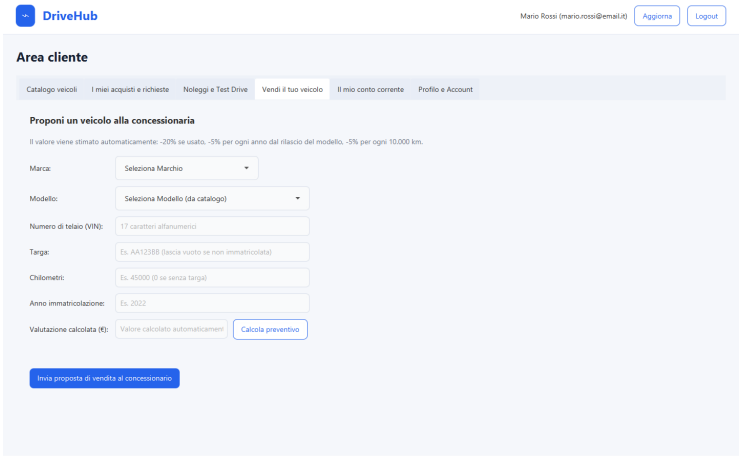


Figura 2.8: Modulo di proposta di un veicolo alla concessionaria.

Conto dimostrativo

La scheda mostra IBAN e saldo del conto usato per gli addebiti e gli accrediti delle operazioni. I comandi di deposito e prelievo modificano soltanto il conto interno al prototipo.

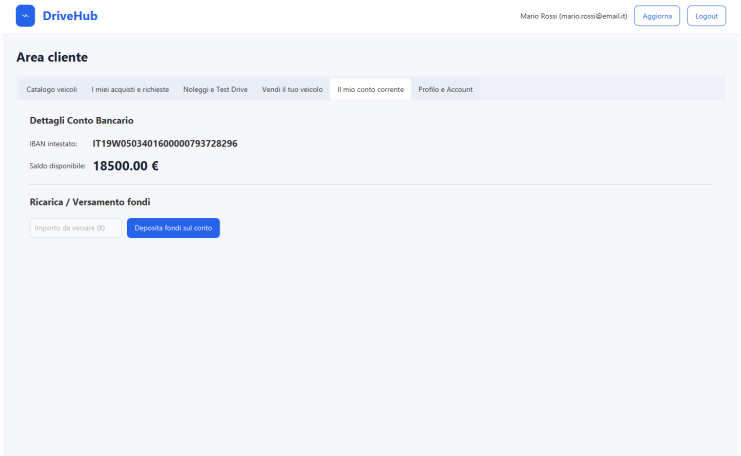


Figura 2.9: Conto dimostrativo del Customer.

Profilo

Il Customer può aggiornare email e password o eliminare il proprio account, confermando le modifiche con la password corrente.

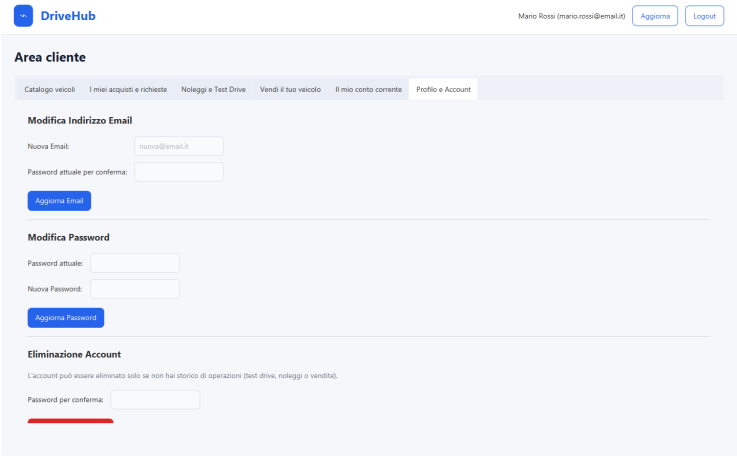


Figura 2.10: Gestione del profilo Customer.

2.3.3 Area Salesman

Veicoli in attesa

La prima scheda raccoglie i veicoli proposti dai clienti e in stato `WAITING`. Il Salesman può approvare la proposta, liquidando il cliente e mettendo il veicolo in vendita, oppure rifiutarla.

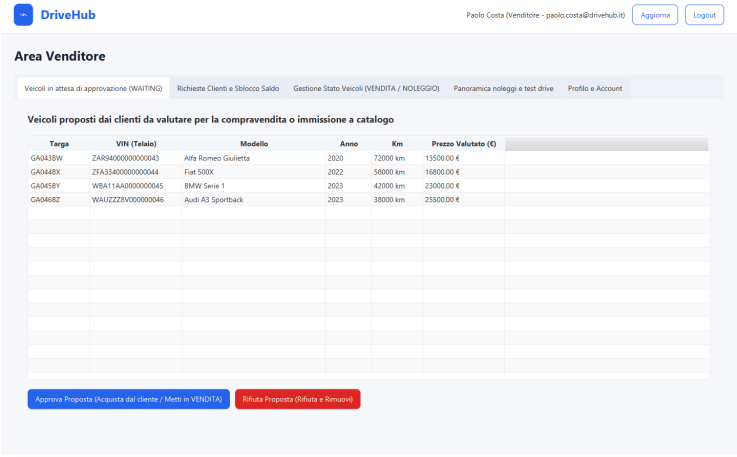


Figura 2.11: Coda delle proposte nell'area Salesman.

Richieste di acquisto

Il Salesman consulta gli acconti versati e autorizza le richieste d'acquisto. L'autorizzazione assegna il venditore e fissa la scadenza del saldo a un mese.

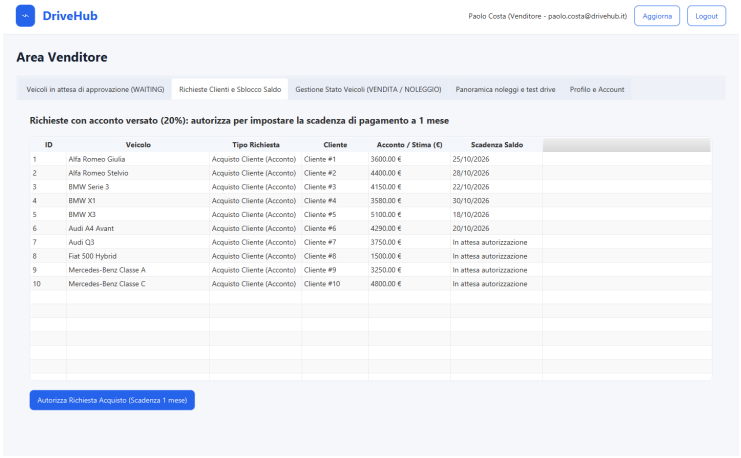


Figura 2.12: Richieste di acquisto e autorizzazione del saldo.

Stato dei veicoli

Questa scheda consente di cambiare la modalità di un veicolo fra vendita e noleggio. Il passaggio da noleggio a vendita comporta l'annullamento dei noleggi futuri e il relativo rimborso.

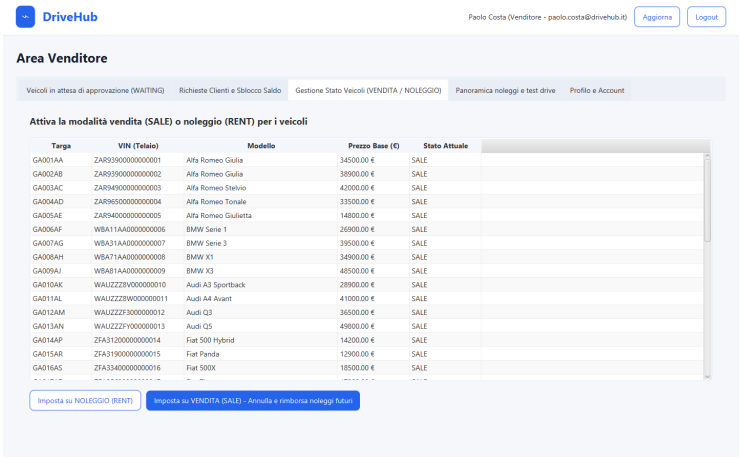


Figura 2.13: Gestione dello stato dei veicoli.

Panoramica prenotazioni

Il Salesman vede le prenotazioni di noleggio e di test drive. La schermata fornisce una vista operativa delle richieste registrate, senza introdurre ulteriori stati di lavorazione.

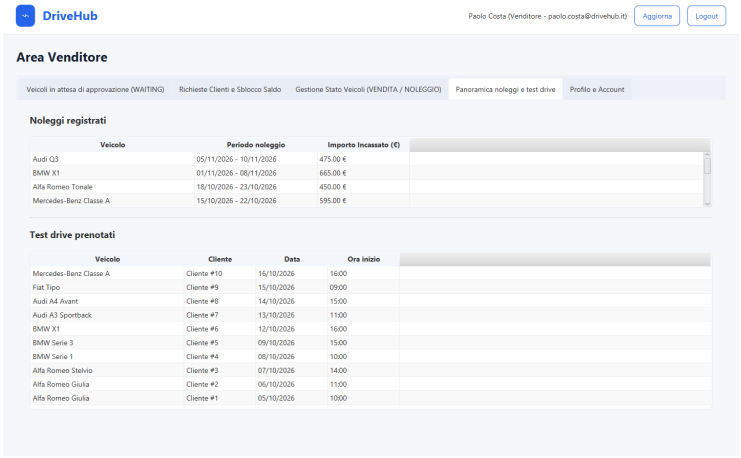


Figura 2.14: Panoramica di noleggi e test drive.

Profilo

Anche il Salesman dispone dei comandi per cambiare email e password e per eliminare l'account.

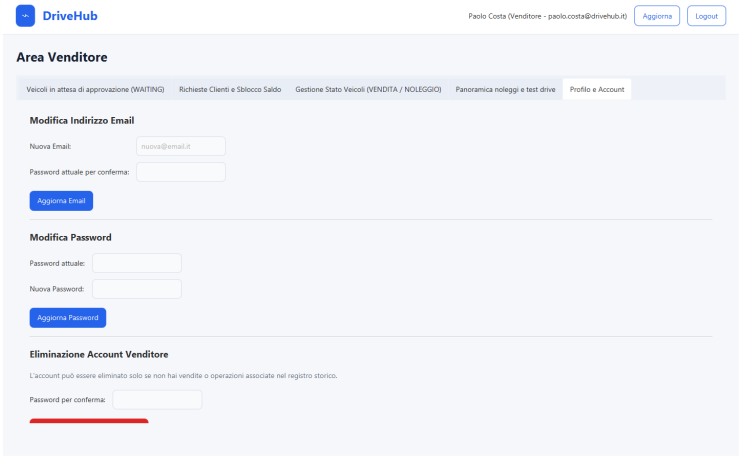


Figura 2.15: Gestione del profilo Salesman.

2.3.4 Area Manager

Parco veicoli

Il Manager consulta tutti i veicoli e può applicare o rimuovere uno sconto sul singolo esemplare. Nella stessa scheda può aggiungere un nuovo veicolo al catalogo o eliminare quello selezionato.

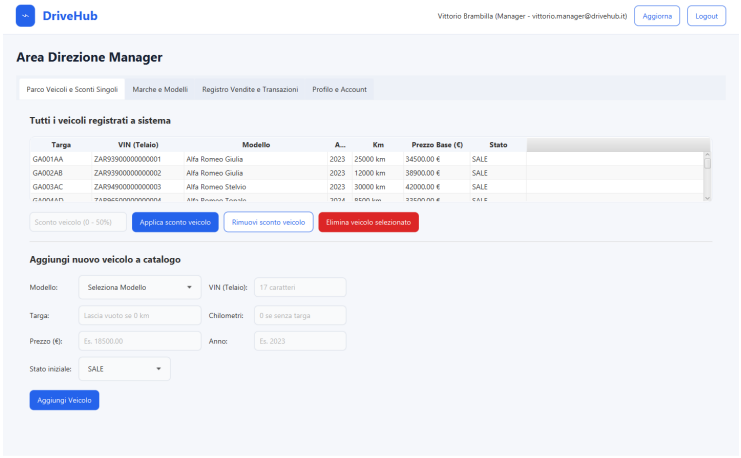


Figura 2.16: Parco veicoli e sconti individuali.

Marche e modelli

La schermata permette di mantenere l'anagrafica delle marche e dei modelli e di impostare sconti a livello di modello, distinti da quelli del singolo veicolo.

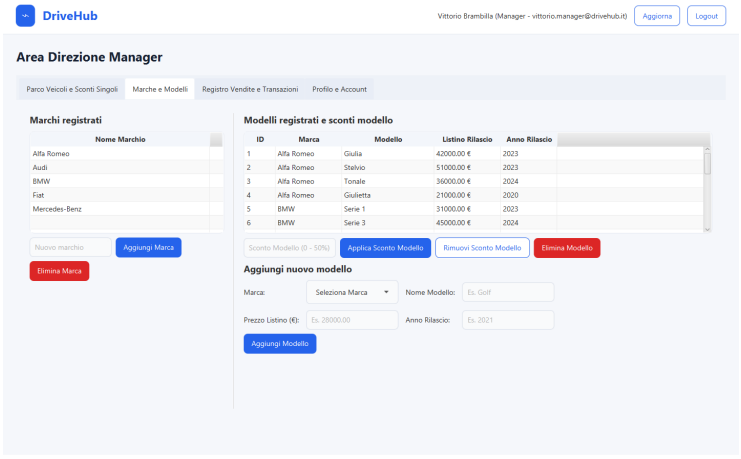


Figura 2.17: Gestione di marche, modelli e sconti.

Registro delle operazioni

Il registro raccoglie vendite concluse, acquisizioni di veicoli dai clienti e penali per acquisti annullati o scaduti. Importo e tipo di operazione sono mostrati nella stessa tabella.

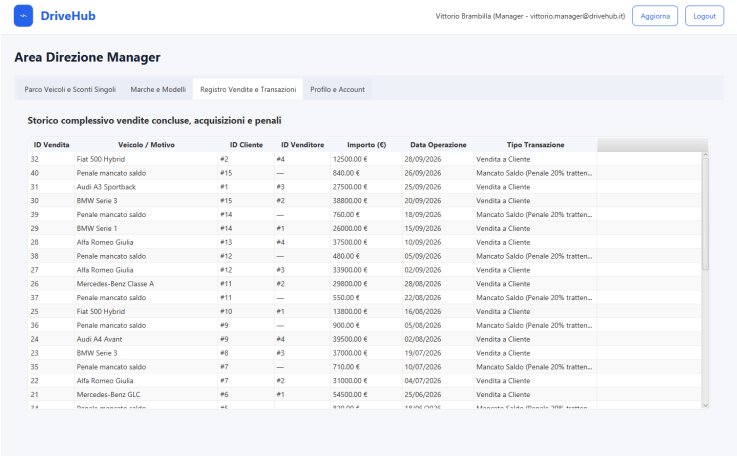


Figura 2.18: Registro delle vendite e delle altre operazioni economiche.

Profilo

Il Manager può modificare le proprie credenziali e gestire l'account dalla scheda dedicata.

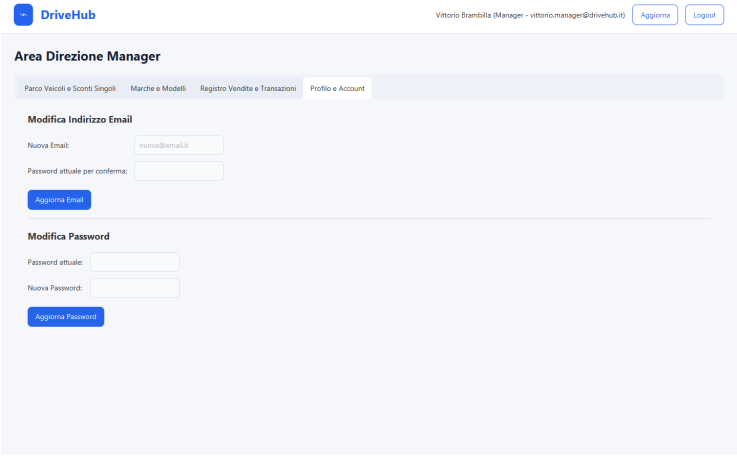


Figura 2.19: Gestione del profilo Manager.

2.4 Page Navigation Diagram

Il diagramma mostra i percorsi tra la pagina iniziale, l'accesso, la registrazione e le aree dei tre ruoli. Dopo l'autenticazione, il sistema apre l'area corrispondente al ruolo dell'utente. I riquadri sotto Customer, Salesman e Manager rappresentano le schede interne alle rispettive aree; il logout riporta alla pagina iniziale.

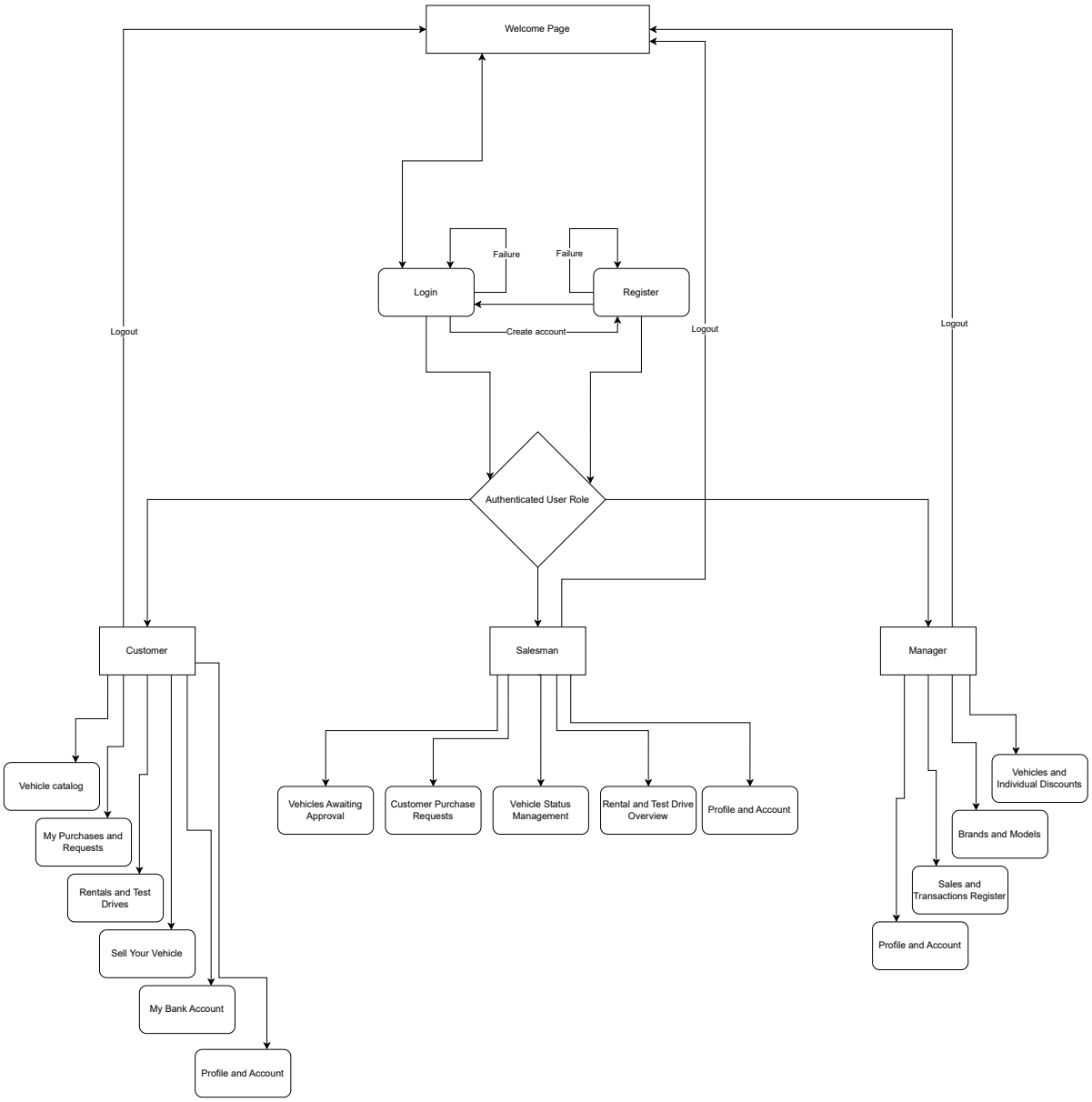


Figura 2.20: Diagramma di navigazione delle pagine e delle schede di DriveHub.

2.5 Package Diagram

Il codice di DriveHub è organizzato sotto il package radice `it.unifi.ing.drivehub` in quattro parti principali. `presentation` gestisce le schermate JavaFX, i controller e la navigazione; `business` coordina i casi d'uso e le strategie di calcolo; `dao` definisce le interfacce di persistenza e le implementa tramite JDBC; `domain` raccoglie le entità e le regole proprie dei veicoli e delle operazioni. Le frecce del diagramma indicano la direzione delle dipendenze: i servizi usano le interfacce DAO, l'implementazione PostgreSQL usa quelle interfacce e le entità, mentre il dominio non dipende dagli altri tre livelli.

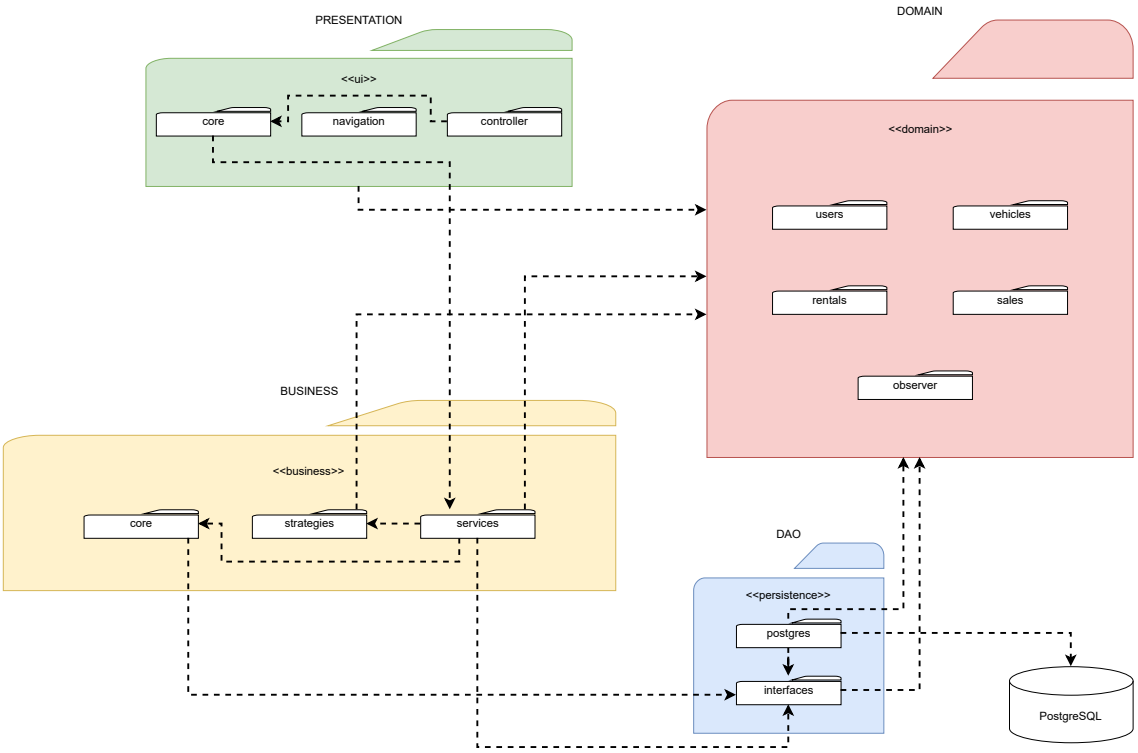


Figura 2.21: Diagramma dei package di DriveHub e delle loro dipendenze.

Il dominio non dipende dagli altri livelli

Le classi del package `domain` non importano classi di `presentation`, `business` o `dao`: le entità non conoscono né le schermate né il modo in cui vengono salvate. Nel diagramma le dipendenze entrano nel dominio, senza frecce in uscita verso gli altri livelli. Questa separazione permette di applicare e verificare le regole delle entità indipendentemente dall'interfaccia JavaFX e da PostgreSQL.

La separazione ha un effetto concreto in `Vehicle`: i metodi `approve`, `block` e `unblock` controllano lo stato corrente prima di modificarlo, senza affidare questa regola al controller o al DAO. Anche `BankAccount` verifica nel dominio che un prelievo non superi il saldo disponibile. I servizi coordinano queste operazioni; le condizioni proprie delle entità restano nelle classi che le rappresentano. Nel disegno le frecce `controller` → `presentation.core` e `presentation.core` → `business.services` rispecchiano il passaggio dai controller a `UiGateway` e poi a `ServiceUiGateway`, che invoca i servizi.

2.6 Class Diagram

Il diagramma dei package della sezione 2.5 è dettagliato in quattro diagrammi di classe. Le figure mantengono i colori dei livelli e riportano i membri essenziali alla lettura delle dipendenze.

2.6.1 Domain Model

Il dominio comprende catalogo, utenti, operazioni ed eventi di inventario. `Vehicle` controlla alcune transizioni di stato e `BankAccount` verifica il saldo nei prelievi. `User`, `Vehicle`, `Request`, `Sale`, `Rent` e `TestDrive` derivano da `BaseEntity`; nel disegno compaiono solo tre generalizzazioni per evitare sovrapposizioni. I riferimenti fra entità usano ID, non associazioni Java dirette. `Vehicle.transitionTo` consente anche cambi di stato generici.

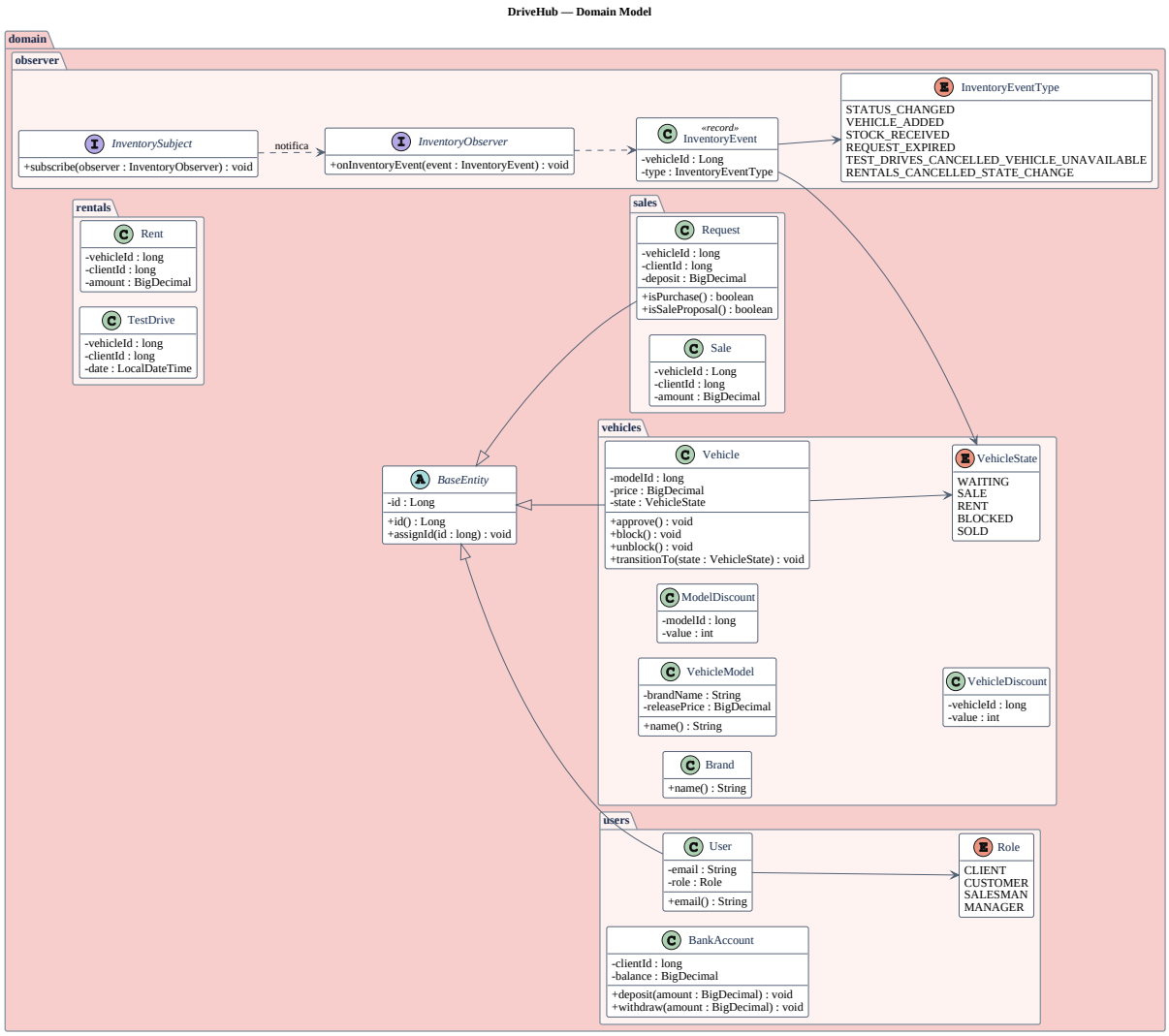


Figura 2.22: Domain Model di DriveHub: classi rappresentative dei package interni al dominio.

2.6.2 Business Layer

I servizi coordinano i casi d'uso dell'applicazione. `TransactionRunner` apre una `UnitOfWork` tramite `DaoFactory`, così le operazioni correlate possono condividere una transazione. `PricingService` delega i calcoli a `PricingStrategy`, implementata da `StandardPricingStrategy`; `AuthService` usa `PasswordHasher` per le credenziali. `SalesService` e `RentalService` implementano `InventorySubject` e notificano gli `InventoryObserver`. Nel diagramma le piccole aree blu e rosse mostrano le interfacce degli altri livelli usate dal business.

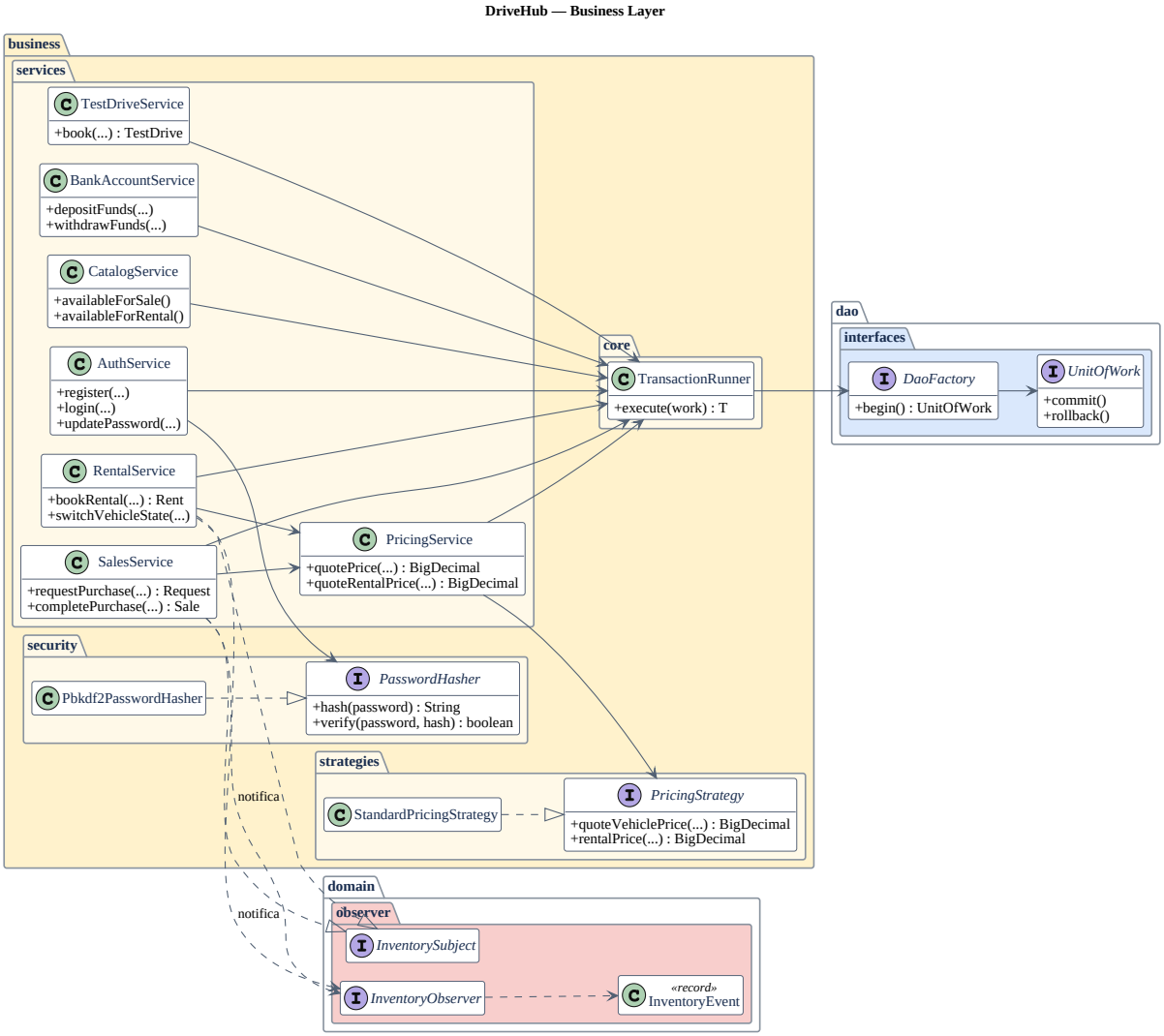


Figura 2.23: Business Layer: servizi, transazioni, Strategy e Observer.

2.6.3 DAO

Il DAO separa le porte di persistenza dalle classi che eseguono le query JDBC. `PostgresDaoFactory.begin` crea una `PostgresUnitOfWork`; questa espone i DAO che condividono la connessione e offre `commit` e `rollback`. Il diagramma mostra quattro coppie interfaccia–implementazione come esempio. Le altre sei porte, per conti, marchi, modelli, sconti, test drive e vendite, seguono lo stesso schema. `JdbcEntityLoader` collabora alla ricostruzione degli oggetti dai risultati SQL.

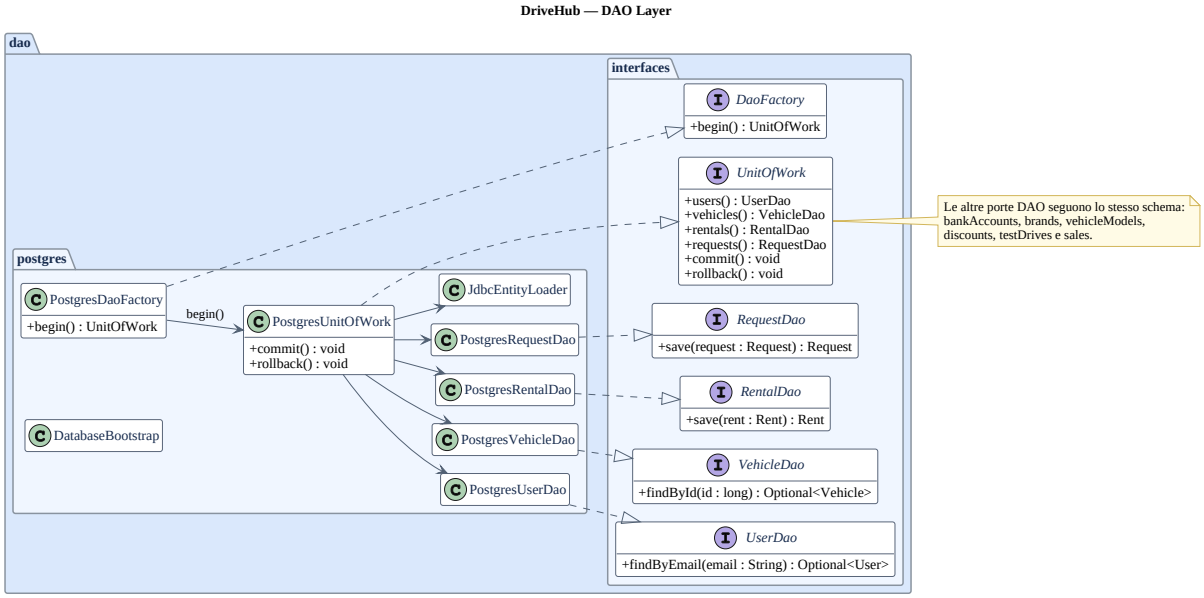


Figura 2.24: DAO: porte, factory, unità di lavoro e implementazioni PostgreSQL rappresentative.

2.6.4 Presentation Layer

La presentazione raccoglie i controller JavaFX, il contesto applicativo, il gateway e la navigazione. I controller dei tre ruoli, del login e della registrazione estendono `AbstractController`, che riceve `ApplicationContext` e permette l'accesso a `UiGateway` e alla sessione. `ServiceUiGateway` implementa il gateway, invoca i servizi e restituisce i record di `UiModels`. `NavigationManager` carica le schermate individuate da `Route` e passa il contesto ai controller; `WelcomeController` è una classe separata, senza ereditarietà da `AbstractController`. Il controllo dei ruoli in `Route` riguarda la navigazione e non costituisce, da solo, un'autorizzazione di tutte le operazioni applicative.

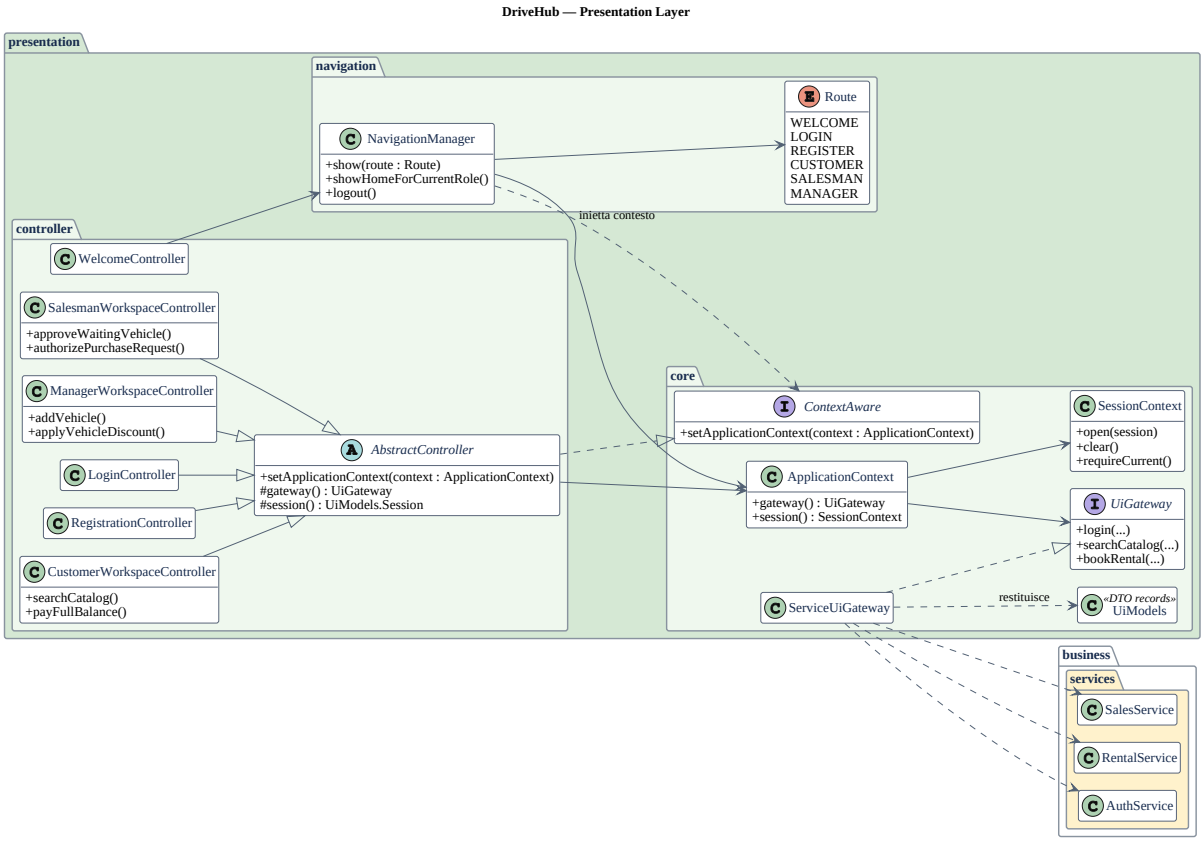


Figura 2.25: Presentation Layer: controller, gateway, sessione e navigazione.

2.7 Sequence Diagram

I diagrammi di sequenza descrivono tre flussi significativi di DriveHub e mostrano come presentazione, servizi, dominio e persistenza collaborano nel tempo. Le chiamate ai servizi partono dai controller JavaFX e attraversano `ServiceUiGateway`: nel diagramma dell'acquisizione il gateway ha una linea di vita, mentre negli altri due è indicato in una nota. La linea «UnitOfWork / DAO» raccoglie l'apertura della transazione tramite `DaoFactory`, le chiamate ai DAO sulla stessa connessione e il commit. I diagrammi sono stati disegnati selezionando le operazioni effettive nei metodi di servizio, non generati automaticamente dai sorgenti.

2.7.1 Acquisto di un veicolo

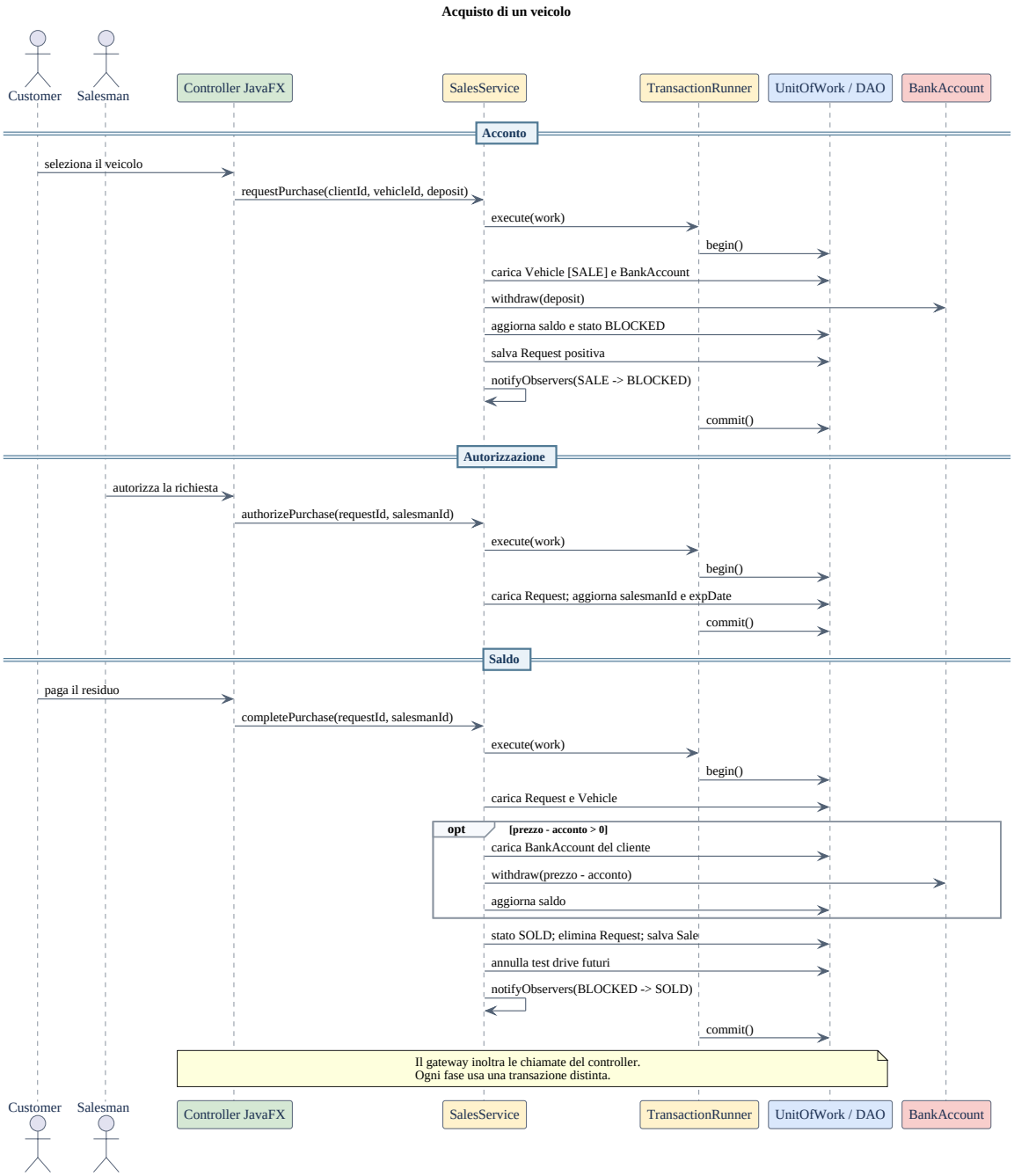


Figura 2.26: Acquisto: acconto del Customer, autorizzazione del Salesman e saldo finale.

La figura 2.26 distingue tre transazioni. La prima verifica lo stato `SALE`, addebita l'acconto sul conto dimostrativo, porta il veicolo a `BLOCKED` e salva una `Request` positiva. Il Salesman imposta poi il proprio ID e una scadenza di un mese. Infine `SalesService` addebita al cliente della richiesta il residuo (prezzo meno acconto), registra la `Sale`, elimina la `Request` e annulla i test drive futuri prima del commit. Il controller abilita il saldo dopo l'autorizzazione, ma `completePurchase` non ricontrolla scadenza né identità del chiamante.

2.7.2 Prenotazione di un noleggio

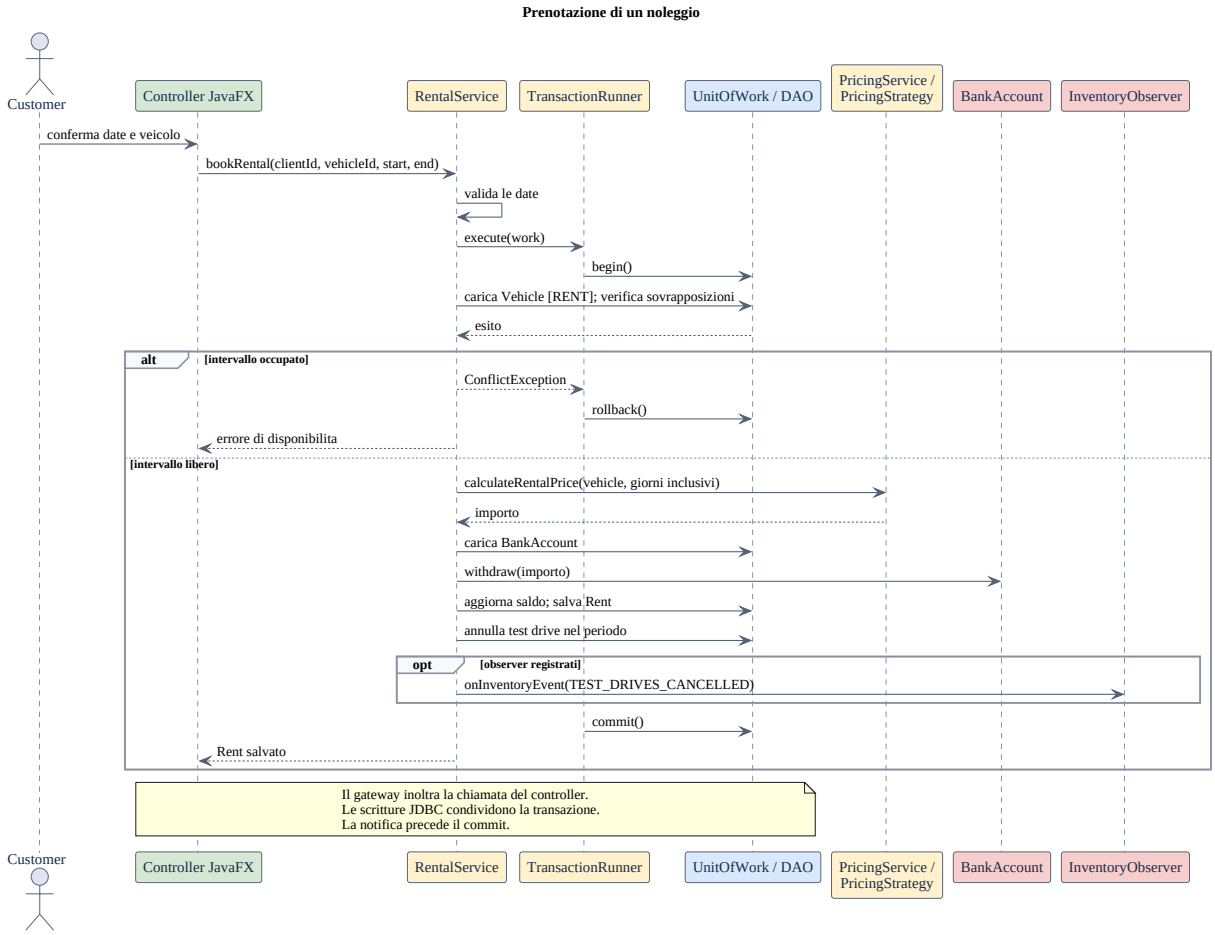


Figura 2.27: Noleggio: controllo delle date, prezzo, addebito e registrazione nella stessa transazione.

La figura 2.27 segue `RentalService.bookRental`. Il servizio valida le date, richiede che il veicolo sia in stato `RENT` e interroga il DAO per cercare noleggi sovrapposti. Se trova un conflitto, `TransactionRunner` esegue il rollback e la prenotazione non viene salvata. Nel percorso valido `PricingService` usa la `PricingStrategy` per calcolare l'importo sui giorni inclusivi; quindi il servizio addebita il conto dimostrativo, salva `Rent` e cancella i test drive nel periodo. La notifica a eventuali `InventoryObserver` precede il commit e non ha la stessa atomicità delle scritture JDBC. Il controllo di sovrapposizione è applicativo: nello schema non è presente un vincolo che escluda due prenotazioni concorrenti sullo stesso intervallo.

2.7.3 Acquisizione di un veicolo del Customer

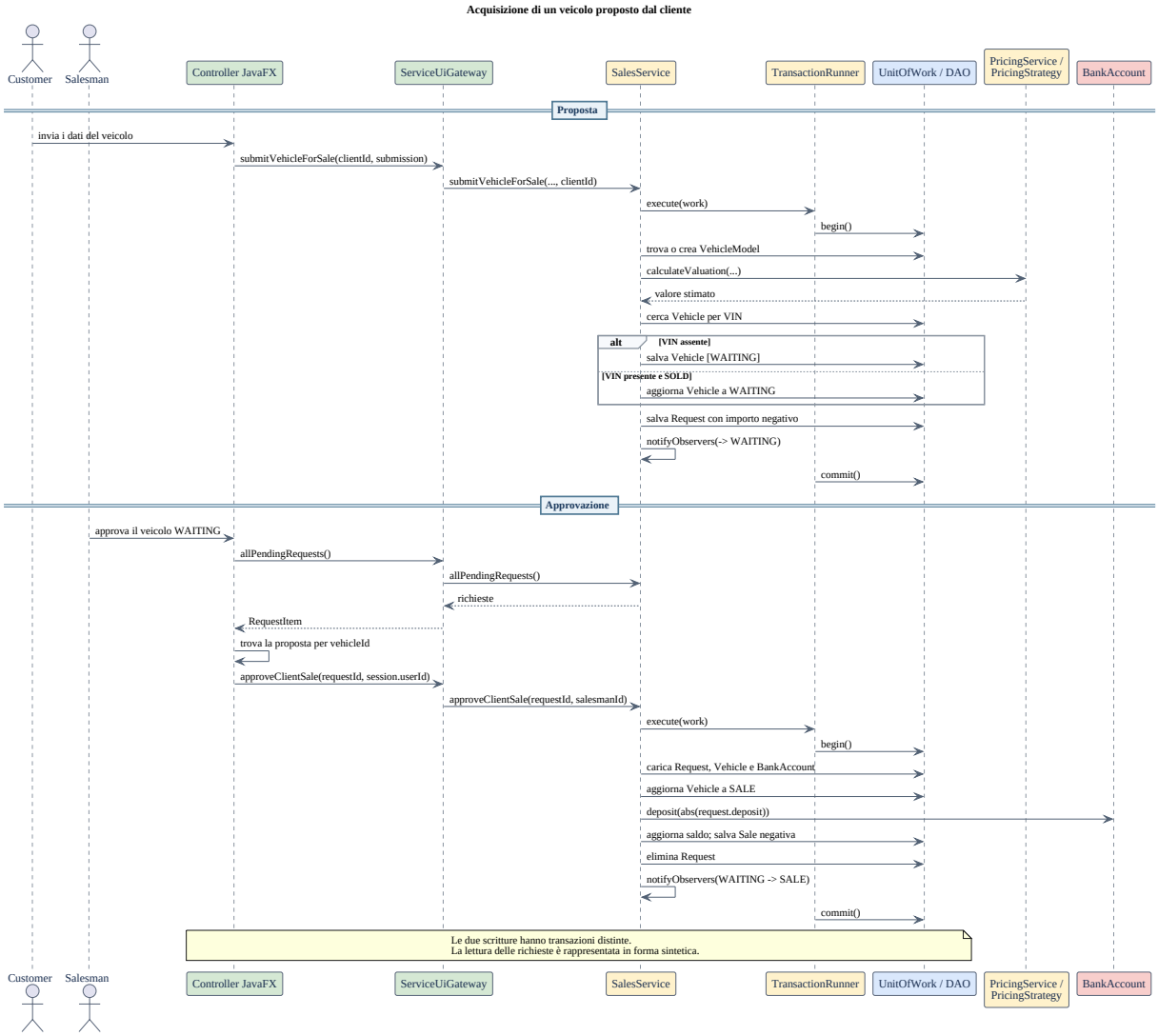


Figura 2.28: Acquisizione: proposta del Customer e approvazione del Salesman, ciascuna in una transazione di scrittura.

Nella figura 2.28, la proposta crea o riusa un veicolo `SOLD`, lo pone in `WAITING` e salva una `Request` negativa. Il controller del Salesman cerca la richiesta di proposta per il veicolo selezionato e passa a `approveClientSale` l'ID del venditore in sessione. L'approvazione accredita il cliente, registra una `Sale` negativa, elimina la richiesta e porta il veicolo a `SALE` in una transazione. Il metodo `approveVehicle` è usato dal controller solo quando non trova una proposta; se chiamato direttamente su una proposta, apre invece due unità di lavoro annidate e usa l'ID venditore costante `1L`. La decisione appartiene al Salesman, non al Manager della specifica storica.

2.8 Database

2.8.1 Schema ER

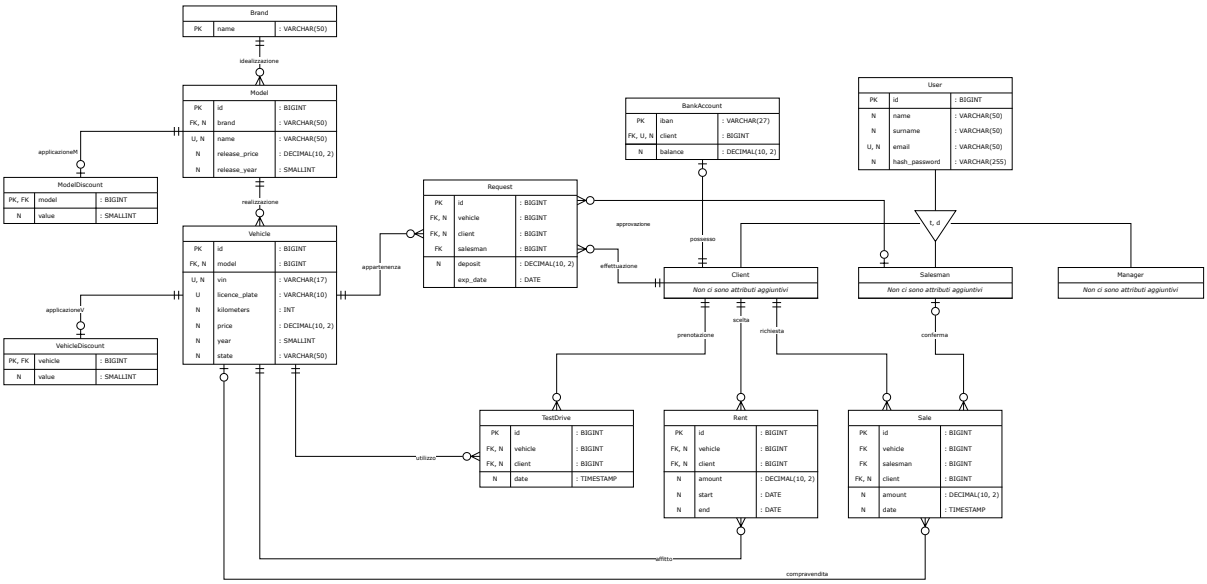


Figura 2.29: Schema entità–relazione di DriveHub, esportato dal diagramma originale dell'autore.

DriveHub conserva catalogo, utenti e operazioni in un database relazionale. La configurazione ordinaria usa PostgreSQL tramite JDBC; la modalità dimostrativa avvia invece H2 in memoria con compatibilità PostgreSQL. Entrambe applicano `init.sql`, ma su H2 `SqlScriptRunner` omette le istruzioni procedurali specifiche di PostgreSQL, fra cui i trigger. `DatabaseBootstrap` registra l'applicazione dello script in `drivehub_schema_migrations`, così la migration non viene rieseguita a ogni avvio. Il seed dimostrativo è configurabile ed è attivo per impostazione predefinita. Il conto dei clienti è un saldo simulato nel database: non rappresenta una connessione a un istituto bancario.

La figura 2.29 riproduce la pagina «schema er» del file originale `docs/SWE ER.drawio`. I simboli alle estremità delle relazioni indicano cardinalità e partecipazione; nelle tabelle, `PK`, `FK`, `U` e `N` indicano rispettivamente chiave primaria, chiave esterna, unicità e valore non nullo. Il disegno combina la generalizzazione concettuale degli utenti con gli attributi e le chiavi delle relazioni persistenti.
`Brand` è associata a zero o più `Model`, e ciascun modello identifica zero o più `Vehicle`. Gli sconti di modello e veicolo sono facoltativi e al più uno per elemento. Un `Client` può effettuare più richieste, test drive, noleggi e compravendite. Ogni `Request`, `TestDrive` e `Rent` si riferisce a un cliente e a un veicolo; il `Salesman` di una richiesta o vendita è facoltativo. In `Sale` è facoltativo anche il veicolo, perché una penale viene registrata senza veicolo. Il medesimo veicolo può comparire in più vendite nel tempo, anche dopo un'acquisizione o una nuova messa in vendita. Il collegamento `Client`–`BankAccount` indica `1..1` sul lato `Client` e `0..1` sul lato `BankAccount`, in accordo con lo schema fisico: ogni conto appartiene a un cliente, mentre un cliente può avere al massimo un conto.

2.8.2 Modello relazionale

La traduzione delle relazioni uno-a-molti pone la chiave esterna sul lato «molti». La generalizzazione `User` è realizzata con tre tabelle distinte, una per ruolo, che ripetono gli attributi comuni. Lo schema fisico di `init.sql` comprende le tredici relazioni seguenti; la vista `user` non è una quattordicesima tabella.

```
brand(name)

model(id, brand, name, release_price, release_year)
    UNIQUE(name)

model_discount(model†, value)

vehicle(id, model, vin, licence_plate*,
    kilometers, price, year, state)
    UNIQUE(vin), UNIQUE(licence_plate)

vehicle_discount(vehicle†, value)

client(id, name, surname, email, hash_password)
    UNIQUE(email)

salesman(id, name, surname, email, hash_password)
    UNIQUE(email)

manager(id, name, surname, email, hash_password)
    UNIQUE(email)

bank_account(iban, client, balance)
```

```
    UNIQUE(client)

request(id, vehicle, client,
    salesman*, deposit, exp_date*)

test_drive(id, vehicle, client, date)

rent(id, vehicle, client, amount, start, end)

sale(id, vehicle*, salesman*,
    client, amount, date)
```

Le chiavi esterne si riferiscono alle seguenti relazioni:

```
model.brand → brand.name
model_discount.model → model.id
vehicle.model → model.id
vehicle_discount.vehicle → vehicle.id
bank_account.client → client.id
request.vehicle, test_drive.vehicle, rent.vehicle, sale.vehicle →
vehicle.id
request.client, test_drive.client, rent.client, sale.client →
client.id
request.salesman, sale.salesman → salesman.id
```

Legenda della notazione

attributo: chiave primaria; attributo: chiave esterna.

†: attributo che è contemporaneamente chiave primaria ed esterna.

*: attributo opzionale (`NULL` ammesso); gli altri sono obbligatori.

UNIQUE(attributo): vincolo di unicità limitato alla relazione indicata.

Le date di `test_drive` e `sale` sono `TIMESTAMP`; `rent` usa `DATE`. Lo schema definisce inoltre `CHECK` su prezzi di modelli e veicoli, saldo, importo del noleggio, sconti, stato del veicolo e ordine delle date del noleggio. Non definisce un `CHECK` per `request.deposit` o `sale.amount`: i due campi usano anche valori negativi per distinguere operazioni diverse. Le eccezioni e i limiti di questi vincoli sono spiegati nel paragrafo seguente.

2.8.3 Utenti, vincoli e semplificazioni

Nel diagramma `User` raccoglie concettualmente nome, cognome, email e hash della password; `Client`, `Salesman` e `Manager` sono i tre ruoli. La traduzione SQL non crea una tabella `user`: replica gli attributi comuni nelle tre tabelle di ruolo e definisce una vista `user` mediante `UNIONALL`, aggiungendo il campo `role`. `PostgresUserDao` scrive nella tabella del ruolo e legge dalla vista quando serve una ricerca comune. Gli `id` sono generati separatamente nelle tre tabelle e possono quindi coincidere: per identificare un utente fra ruoli occorre considerare anche `role`; l'`id` mostrato nel supertipo non è una chiave primaria globale della vista.

La registrazione applicativa verifica che l'email non sia già utilizzata e, per un `Client`, crea il `BankAccount` nella stessa transazione. La relazione uno-a-uno mostrata nel diagramma descrive questo percorso ordinario: nel database `UNIQUE(client)` impedisce più conti per lo stesso cliente, ma non obbliga ogni cliente ad averne uno. Per facilitare le

prove nella modalità dimostrativa, i trigger sull'email non vengono installati su H2. Su PostgreSQL `init.sql` definisce invece la funzione `check_user_email_disjoint` e tre trigger, ma il loro confronto esclude righe con lo stesso `id` numerico anche se provengono da tabelle diverse. Gli ID dei ruoli sono indipendenti, quindi i trigger non assicurano l'unicità trasversale in tutti i casi. `UNIQUE(email)` vale all'interno di ciascuna tabella; il normale percorso di registrazione esegue anche una ricerca preventiva nella vista `user`.

Le regole espresse nello schema includono, fra le altre, `UNIQUE` su VIN e targa, saldo non negativo, sconti fra 0 e 50, stato del veicolo limitato a `WAITING`, `SALE`, `RENT`, `BLOCKED` e `SOLD`, e fine del noleggio non precedente all'inizio. Altre condizioni restano applicative: non esiste un vincolo SQL che impedisca la sovrapposizione temporale di due `Rent` o renda unico lo slot di `TestDrive`. Il segno di `Request.deposit` distingue l'acconto d'acquisto dalla stima di cessione; il segno di `Sale.amount` distingue la vendita dall'esborso per un'auto acquisita. Una penale è una `Sale` con `vehicle` e `salesman` nulli. Lo schema consente queste rappresentazioni, ma non contiene `CHECK` che ne garantiscano tutte le combinazioni ammissibili.

Capitolo 3

Implementazione

Questo capitolo descrive le scelte implementative dei livelli `domain`, `business`, `presentation` e `dao`, concentrandosi sulle classi e sulle regole che guidano i casi d'uso principali. Gli esempi di codice provengono dai sorgenti di DriveHub; i diagrammi del capitolo precedente ne mostrano le relazioni.

3.1 Domain Model

Il Domain Model rappresenta veicoli, utenti e operazioni senza importare classi di `presentation`, `business` o `dao`. È organizzato in cinque pacchetti, già mostrati nel diagramma di classe della figura 2.22:

- **users**: identità, ruolo e conto dimostrativo del cliente;
- **vehicles**: catalogo, stato dei veicoli e sconti;
- **rentals**: noleggi e prenotazioni di test drive;
- **sales**: richieste, compravendite e penali;
- **observer**: contratti ed eventi relativi all'inventario.

Alla radice di `domain`, `BaseEntity` conserva l'ID delle entità persistite. Una nuova entità nasce con ID nullo; `assignId` accetta soltanto valori positivi e impedisce di sostituire un ID già assegnato con uno diverso. `requireId` segnala invece l'uso di un'entità ancora priva di identità persistente. Le associazioni fra gli oggetti sono rappresentate soprattutto da ID: per esempio `Vehicle` conserva `modelId` e `Request` conserva `vehicleId` e `clientId`. Il caricamento degli oggetti associati spetta ai DAO e ai servizi.

3.1.1 Users

Il pacchetto `users` contiene `User`, `Role` e `BankAccount`. A differenza della gerarchia concettuale nello schema ER, nel codice Java `Client`, `Salesman` e `Manager` non sono sottoclassi: sono valori del campo `role` di un'unica classe `User`. L'enumerazione contiene `CLIENT`, `CUSTOMER`, `SALESMAN` e `MANAGER`; `Role.fromString` converte il vecchio nome `CUSTOMER` in `CLIENT`. La persistenza dei ruoli in tabelle distinte è descritta nel paragrafo 2.8.

Il costruttore di `User` richiede nome, cognome e hash non vuoti; normalizza l'email in minuscolo e ne controlla il formato. La factory `create` produce un utente senza ID, che il DAO assegna dopo l'inserimento. Il controllo di unicità dell'email richiede una ricerca nei dati persistiti: il singolo oggetto non può garantirlo. `equals` e `hashCode` si basano

sull'email, indipendentemente dal ruolo; questa scelta presuppone che il normale flusso applicativo eviti email duplicate fra ruoli.

`BankAccount` non deriva da `BaseEntity`: è identificato dall'IBAN e conserva l'ID del cliente intestatario. Il costruttore richiede un IBAN di 27 caratteri e un saldo non negativo. I metodi `deposit` e `withdraw` ammettono solo importi positivi; il prelievo verifica anche che il saldo sia sufficiente prima di aggiornarlo. Il conto rappresenta un saldo simulato, senza collegamento a un servizio bancario esterno.

3.1.2 Vehicles

Il pacchetto `vehicles` riunisce `Brand`, `VehicleModel`, `Vehicle`, `VehicleState`, `ModelDiscount` e `VehicleDiscount`. `Brand` richiede un nome non vuoto; `VehicleModel` associa marca, nome, prezzo e anno di lancio, rifiutando prezzi negativi e anni fuori dall'intervallo 1900–anno corrente. Gli sconti di modello e di veicolo sono percentuali intere fra 0 e 50 e conservano l'ID del rispettivo elemento.

`Vehicle` conserva il modello, il VIN, l'eventuale targa, chilometraggio, prezzo, anno e stato. Il costruttore normalizza VIN e targa in maiuscolo, rifiuta chilometri e prezzi negativi e impone chilometri zero quando la targa manca. `VehicleState` comprende `WAITING`, `SALE`, `RENT`, `BLOCKED` e `SOLD`. Tre metodi esprimono transizioni con una precondizione esplicita:

```
public void approve() {
    if (state != VehicleState.WAITING) {
        throw new DomainRuleViolationException(
                "Cannot approve vehicle not in WAITING state");
    }
    this.state = VehicleState.SALE;
}

public void block() {
    if (state != VehicleState.SALE) {
        throw new DomainRuleViolationException(
                "Cannot block vehicle not in SALE state");
    }
    this.state = VehicleState.BLOCKED;
}
```

Listing 3.1: Transizioni controllate dello stato di Vehicle.

`unblock` applica la stessa struttura al passaggio da `BLOCKED` a `SALE`. Esiste anche `transitionTo`, che assegna qualsiasi `VehicleState` non nullo, e `updateDetails` riceve direttamente il nuovo stato: dunque la classe non impone a ogni chiamante l'uso delle tre transizioni controllate. I servizi devono verificare il contesto dell'operazione; il database limita i valori ammessi, ma non l'ordine dei passaggi.

3.1.3 Rentals

`Rent` e `TestDrive` condividono il riferimento a un cliente e a un veicolo, ma modellano tempi diversi. `Rent` registra un intervallo di giorni `LocalDate` e un importo non negativo; il costruttore respinge una data finale precedente a quella iniziale. `TestDrive` registra

invece un `LocalDateTime` e verifica che l'ora di partenza sia fra le 09:00 e le 16:00 con minuti pari a zero. Il metodo `end` restituisce l'ora successiva alla partenza.

Nessuna delle due classi contiene uno stato di workflow. Un oggetto `Rent` valido può comunque sovrapporsi a un altro noleggio dello stesso veicolo, e due oggetti `TestDrive` validi possono avere lo stesso slot: sono verifiche che richiedono la consultazione delle prenotazioni già registrate. I servizi eseguono queste ricerche prima del salvataggio; il rischio residuo delle richieste concorrenti è discusso nella sezione 3.4.4.

3.1.4 Sales

Il pacchetto `sales` contiene `Request` e `Sale`. `Request` rappresenta due pratiche con lo stesso insieme di campi. `createPurchase` richiede un acconto positivo per l'acquisto dal catalogo; `createSaleProposal` richiede una stima positiva e la salva con segno negativo per la proposta di cessione del cliente:

```
public static Request createPurchase(long vehicleId, long clientId,
                                     BigDecimal deposit) {
    if (deposit.compareTo(BigDecimal.ZERO) <= 0) {
        throw new IllegalArgumentException(
                "Purchase deposit must be strictly positive");
    }
    return new Request(null, vehicleId, clientId, null, deposit, null);
}

public static Request createSaleProposal(long vehicleId, long clientId,
                                         BigDecimal estimatedValue) {
    if (estimatedValue.compareTo(BigDecimal.ZERO) <= 0) {
        throw new IllegalArgumentException(
                "Estimated value must be strictly positive");
    }
    return new Request(null, vehicleId, clientId, null,
                       estimatedValue.negate(), null);
}
```

Listing 3.2: Factory method di Request per i due tipi di pratica.

Il costruttore vieta un valore zero e `isPurchase`/`isSaleProposal` distinguono le due pratiche dal segno. L'eventuale `salesmanId` e la scadenza sono assegnati in un momento successivo tramite i metodi della richiesta. La codifica compatta evita due tipi persistenti separati, ma rende necessario interpretare correttamente il segno nei servizi e nelle query.

`Sale` è il registro degli esiti economici. Un importo positivo con veicolo rappresenta una vendita al cliente; un importo negativo rappresenta l'acquisizione di un'auto dal cliente. La factory `penalty` costruisce una riga senza veicolo né venditore per la penale. `isAcquisition` controlla il segno e `isPenalty` controlla i due riferimenti nulli. Il costruttore, però, richiede soltanto cliente positivo, importo e data non nulli: non impedisce ogni combinazione incoerente di segno e riferimenti. La coerenza del caso d'uso resta responsabilità dei servizi.

3.1.5 Observer

Il pacchetto `observer` definisce `InventorySubject`, `InventoryObserver`, il record `InventoryEvent` e l'enumerazione `InventoryEventType`. L'evento trasporta tipo, veicolo interessato, targa, eventuale stato precedente e successivo, e istante di creazione. Le factory `statusChanged` e `requestExpired` preparano i dati degli eventi più ricorrenti.

Le interfacce appartengono al dominio, mentre i soggetti concreti sono `SalesService` e `RentalService` nel livello business. Questi servizi registrano gli observer e li notificano quando un'operazione modifica l'inventario. Il record descrive il fatto avvenuto, senza conoscere controller o database; la notifica, tuttavia, è eseguita dal servizio durante l'operazione e non costituisce una garanzia di consegna dopo il commit della transazione.

3.2 Business Layer

Il Business Layer coordina le entità del dominio per eseguire i casi d'uso. I servizi ottengono i dati attraverso le interfacce DAO e delegano alle entità le regole che queste possono verificare autonomamente. Il diagramma della figura 2.23 mostra le dipendenze principali. Il package `business` è suddiviso in sei parti:

- **services**: servizi applicativi per identità, catalogo, conto, test drive, noleggio, vendita e prezzi; contiene anche `RegistrationRequest` e `CatalogModels`;
- **core**: `TransactionRunner`, che esegue un lavoro all'interno di una `UnitOfWork`;
- **strategies**: algoritmi sostituibili per prezzi, noleggi e valutazioni;
- **security**: contratto `PasswordHasher` e implementazione `Pbkdf2PasswordHasher`;
- **banking**: `IbanGenerator` per il conto simulato del cliente;
- **exceptions**: errori applicativi per autenticazione, conflitti e risorse assenti.

La separazione mantiene le query nelle porte DAO, la verifica del saldo in `BankAccount` e la scelta del flusso nei servizi. Il business dipende dal dominio e dalle interfacce di persistenza, non dai controller JavaFX né dalle classi JDBC concrete.

3.2.1 Core: unità di lavoro e transazioni

Ogni servizio costruisce un `TransactionRunner` a partire da `DaoFactory`. `execute` apre un'unità di lavoro, passa la stessa istanza alla lambda del caso d'uso, esegue il commit se il lavoro termina e il rollback se viene lanciata un'eccezione. Il `try`-with-resources chiude l'unità anche in caso di errore:

```
try (UnitOfWork unit = daoFactory.begin()) {
    try {
        T result = work.apply(unit);
        unit.commit();
        return result;
    } catch (RuntimeException | Error failure) {
        try {
            unit.rollback();
        } catch (RuntimeException rollbackFailure) {
            failure.addSuppressed(rollbackFailure);
        }
        throw failure;
    }
}
```

Listing 3.3: Nucleo di TransactionRunner.execute, dal sorgente corrente.

Questo meccanismo lega fra loro le scritture eseguite nella stessa lambda, come saldo e registrazione di un noleggio. Non unifica però due chiamate di servizio separate; inoltre una chiamata annidata a `execute` apre un'altra `UnitOfWork`. L'implementazione JDBC del contratto è descritta nella sezione 3.4.5.

3.2.2 Services: responsabilità

`AuthService` gestisce registrazione, login e modifiche dell'account; `BankAccountService` consulta e aggiorna il saldo del cliente. `CatalogService` legge e modifica marche, modelli e veicoli. `TestDriveService` e `RentalService` verificano disponibilità temporali e salvano le prenotazioni. `SalesService` governa richieste di acquisto, saldo, penali e proposte di cessione. `PricingService` legge modello e sconti e passa i dati alla strategia di prezzo. La classe di supporto `CatalogModels` trova o crea marca e modello durante la proposta di un veicolo; `RegistrationRequest` trasporta i dati necessari alla registrazione.

La distinzione fra validazione locale e verifica applicativa è visibile nel test drive. `TestDrive` valida l'ora contenuta nel proprio oggetto; `TestDriveService.book` deve anche interrogare le prenotazioni già presenti, il noleggio del veicolo e il numero di test drive attivi del cliente. Analogamente `Rent` valida date e importo, mentre `RentalService.bookRental` decide disponibilità, prezzo e addebito. I paragrafi seguenti esaminano questi flussi e le altre decisioni dei servizi.

3.2.3 Services: flussi applicativi

I servizi coordinano le porte DAO entro `TransactionRunner.execute`. Le tre sequenze della sezione 2.7 visualizzano le chiamate principali; qui sono analizzate le decisioni e le scritture, incluse le alternative che un diagramma compatto non rende leggibili.

Catalogo e test drive

`CatalogService.availableForSale`, `availableForRental` e `waitingVehicles` interrogano `VehicleDao.findByState`. `ServiceUiGateway.searchCatalog` applica poi in memoria un testo a descrizione, VIN e targa e calcola il prezzo esposto con `PricingService.quotePrice`. Il filtro viene quindi eseguito in parte dal database e in parte dal gateway. La UI non offre il prezzo massimo promesso da `RF-CAT-01`.

`TestDriveService.book` controlla prima data e ora, poi carica il veicolo. Ammette `SALE` e `RENT`; limita a tre i test drive attivi del cliente; interroga `RentalDao.isVehicleRentedOnDate` e `TestDriveDao.hasConflictingTestDrive`; infine salva `TestDrive`. Lo slot applicativo è un'ora, con partenza alle 09:00–16:00. Il servizio tronca i minuti della `LocalTime` al costruire `dateTime`, quindi un input con minuti diversi da zero viene ricondotto all'inizio dell'ora anziché rifiutato in quel punto. Il costruttore `TestDrive` ammette solo minuti zero.

```
if (unit.rentals().isVehicleRentedOnDate(vehicleId, date)) {
    throw new ConflictException("Il veicolo e noleggiato per il giorno
        selezionato");
}
if (unit.testDrives().hasConflictingTestDrive(vehicleId, dateTime)) {
    throw new ConflictException("La fascia oraria e gia occupata");
}
TestDrive testDrive = TestDrive.create(vehicleId, clientId, dateTime);
return unit.testDrives().save(testDrive);
```

Listing 3.4: Controlli sequenziali di disponibilità nel test drive, estratto da `TestDriveService.book`.

Nel listato le stringhe di errore sono abbreviate per mostrare le chiamate rilevanti; per i messaggi letterali si rimanda al sorgente. Le due letture non bloccano un inserimento concorrente: l'assenza di una unique su `(vehicle,date)` è discussa in sezione 3.4.4.

Noleggio con saldo del conto

La UI mostra un preventivo per `days=DAYS.between(start,end)+1`. Alla conferma, `RentalService.bookRental` ripete i controlli e ricalcola l'importo. L'algoritmo verifica stato `RENT` e overlap, poi trova il conto del cliente, esegue `BankAccount.withdraw`, salva il nuovo saldo, inserisce `Rent` e cancella test drive nel periodo. Tutte le scritture DAO passano sulla stessa `PostgresUnitOfWork`. Non esiste un pagamento con stato `PENDING`/`COMPLETED`/`FAILED`: `RF-PAY-01` è quindi coperto soltanto dall'addebito dimostrativo.

La query di overlap usa gli estremi inclusivi. Per prenotazioni già salvate dello stesso veicolo, `NOT("end"<?ORstart>?)` rileva un'intersezione. Due periodi adiacenti che condividono un giorno sono conflittuali; uno che inizia il giorno dopo la fine dell'altro è ammesso. Lo schema protegge `end>=start` ma non l'assenza di sovrapposizioni, perciò il controllo non è sicuro sotto concorrenza.

```
SELECT 1 FROM "rent"
WHERE vehicle = ? AND NOT ("end" < ? OR start > ?)
LIMIT 1
```

Listing 3.5: Predicato di overlap da `PostgresRentalDao.hasOverlappingRentals`.

`RentalService.switchVehicleState` è un secondo flusso significativo. Quando un Salesman passa un veicolo da `RENT` a `SALE`, il servizio legge noleggi futuri, riaccredita l'intero importo sui conti dei clienti e cancella le righe `rent`; quindi cambia lo stato del veicolo. L'operazione è in una transazione JDBC, ma le notifiche Observer vengono emesse durante l'esecuzione della lambda, prima del commit. Nel verso `SALE` a `RENT` il codice non compie il medesimo rimborso.

Richiesta di acquisto, saldo e penale

Il controller calcola l'acconto come 20% di `selectedCatalogVehicle.price()` e chiede conferma. `SalesService.requestPurchase` riceve però l'importo dal chiamante senza confrontarlo con `PricingStrategy.calculateDeposit`: un client alternativo può passare un valore positivo diverso. Il metodo controlla `VehicleState.SALE`, sottrae l'acconto dal conto, aggiorna il veicolo a `BLOCKED` e salva `Request.createPurchase`. `authorizePurchase` aggiunge `salesmanId` e `expDate=LocalDate.now().plusMonths(1)`.

`CustomerWorkspaceController.payFullBalance` nasconde il comando finché `expDate` è nulla; il controllo però non è ripetuto in `SalesService.completePurchase`. Quest'ultimo calcola `remaining=vehicle.price()-request.deposit()`, preleva il residuo se positivo, imposta `SOLD`, elimina la richiesta, salva `Sale` e cancella i futuri test drive. Il prezzo di partenza del saldo è il campo `Vehicle.price`, mentre il catalogo mostra `PricingService.quotePrice`; il preventivo non è conservato nella `Request`. Questa incoerenza implementativa può far pagare un totale diverso dal prezzo mostrato, come illustrato nella sezione 3.2.4.

Se una richiesta scade o viene annullata, `cancelRequestWithPenalty` calcola una penale del 20% dell'acconto con `HALF_UP`, rimborsa il restante 80%, cancella la richiesta, riporta il veicolo in `SALE` e salva una `Sale.penalty`. `processExpiredRequests` ricerca le pratiche scadute e chiama quel metodo separatamente per ognuna; cattura e ignora le eccezioni della singola pratica. L'elaborazione avviene all'accesso e in alcune letture del gateway, non tramite scheduler persistente.

Proposta di cessione del veicolo

`SalesService.submitVehicleForSale` usa `CatalogModels.findOrCreate` per marca e modello, quindi `PricingService.calculateValuation`. Un VIN già presente è ammesso soltanto se il veicolo precedente è `SOLD`; in quel caso la riga viene aggiornata a `WAITING`, altrimenti viene creata una nuova riga. `Request.createSaleProposal` memorizza il valore stimato con segno negativo. La proposta non attraversa un'offerta modificabile dal Salesman e una successiva approvazione Manager come nella specifica storica.

`approveClientSale` porta il veicolo a `SALE`, accredita al cliente il valore assoluto della `Request`, registra una `Sale` negativa ed elimina la richiesta. `rejectClientSale` elimina la richiesta e conserva il veicolo come `SOLD` solo se esistono vendite pregresse, altrimenti lo elimina. Quando trova la proposta, il controller del Salesman chiama direttamente `approveClientSale` passando l'ID della sessione; usa `approveVehicle` se non trova alcuna proposta per il veicolo selezionato. Se `approveVehicle` viene chiamato direttamente su una proposta, chiama `approveClientSale` all'interno di un'altra lambda transazionale, aprendo una seconda unità di lavoro, e passa il valore costante `1L` come ID del Salesman. Questa scorciatoia può attribuire la vendita al venditore sbagliato quando opera un altro account. Il confine annidato è analizzato in sezione 3.4.5.

La stima esposta prima della proposta può divergere dal valore poi salvato: `quoteValuation` usa un prezzo di rilascio predefinito di 25.000 quando non trova il modello, mentre `CatalogModels.findOrCreate` usa l'importo fornito o 20.000 per creare il nuovo modello. Sono due fallback diversi nei sorgenti, non una policy documentata da un requisito.

3.2.4 Strategies: prezzi e sconti

`PricingService` dipende da `PricingStrategy`; la composizione normale istanzia `StandardPricingStrategy`. La porta rende sostituibile l'algoritmo nei test o in una futura revisione. La persistenza offre al massimo uno sconto per modello e uno per veicolo; `quotePrice` sceglie il maggiore fra i due valori, non li cumula. `ModelDiscount` e `VehicleDiscount` ammettono interi fra 0 e 50, coerenti con i CHECK SQL.

Per un veicolo in catalogo, la strategia parte da `VehicleModel.releasePrice`; se la targa non è nulla applica un fattore 0,80, poi $1-p/100$ con p pari allo sconto promozionale maggiore, quindi $0{,}95^{a}$ per gli anni trascorsi e $0{,}95^{\lfloor km/10000 \rfloor}$ per il chilometraggio. Il risultato è arrotondato a due decimali con `HALF_UP` e non scende sotto zero. La formula è una scelta del codice e non va confusa con un prezzo di mercato verificato:

$$P_{\text{catalogo}} = \text{round}_2\big(P_{\text{lancio}} \cdot f_{\text{targa}} \cdot (1 - p/100) \cdot 0{,}95^{a+\lfloor km/10000 \rfloor}\big), \qquad f_{\text{targa}} \in \{1; 0{,}80\}.$$

`calculateDeposit(price)` restituisce il 20% del prezzo passato dal chiamante. Nel flusso GUI l'acconto è calcolato sul prezzo mostrato; `SalesService.requestPurchase`, tuttavia, non ricontrolla la percentuale.

Anomalia implementativa nel completamento dell'acquisto. Il prezzo mostrato nel catalogo e quello usato per il saldo derivano da due calcoli diversi: `ServiceUiGateway.searchCatalog` usa `PricingService.quotePrice`, mentre `SalesService.completePurchase` legge direttamente `Vehicle.price`. Il secondo metodo non applica gli sconti promozionali e non recupera il preventivo accettato dal cliente. La differenza può emergere anche senza promozioni, poiché `quotePrice` considera prezzo di lancio, targa, età e chilometraggio.

Per esempio, con prezzo di lancio e `Vehicle.price` entrambi a 40.000 euro, veicolo dell'anno corrente senza targa, a zero chilometri e promozione del 10%, il catalogo mostra 36.000 euro e la UI calcola un acconto di 7.200 euro. Al completamento, il servizio addebita però 32.800 euro anziché 28.800: il totale pagato è 40.000 euro. È quindi un'incoerenza del codice rispetto al prezzo presentato al cliente, non una regola commerciale del sistema. La relazione documenta il comportamento presente; l'implementazione non è stata modificata.

Il noleggio usa `Vehicle.price`, non `quotePrice`. Per d giorni inclusivi, `rentalPrice` calcola una tariffa base giornaliera di price/1000, poi un fattore $0{,}95^{d-1}$ sul totale; arrotonda a centesimi e impone un minimo di 0,01. La label della UI in `CustomerWorkspaceController.updateRentalQuotePreview` dice invece «Sconto 5% ogni 5 gg applicato»: il testo non corrisponde alla formula corrente e deve essere corretto o accompagnato da una modifica della policy. La valutazione del veicolo ceduto riusa il fattore targa, età e km, senza promozione Manager.

Tabella 3.1: Origine degli importi nei flussi economici.

Importo	Origine nel codice	Uso
Prezzo esposto nel catalogo	`PricingService.quotePrice`	Anteprima di acquisto
Acconto GUI	20% di `VehicleItem.price`	`requestPurchase`
Saldo effettivo	`Vehicle.price-Request.deposit`	`completePurchase`

Importo	Origine nel codice	Uso
Prezzo noleggio	`StandardPricingStrategy.rentalPrice`	`bookRental`
Stima cessione	`calculateValuation`	`submitVehicleForSale`

3.2.5 Security e Banking: autenticazione e conto

`AuthService.register` normalizza l'email, ne verifica l'assenza tramite `UserDao.findByEmail`, calcola l'hash e salva `User`. Se il ruolo è `CLIENT` o `CUSTOMER`, crea nello stesso `UnitOfWork` un `BankAccount` con saldo iniziale zero e IBAN derivato da `IbanGenerator`. `login` cerca l'email e chiama `PasswordHasher.verify`. Il verificatore `Pbkdf2PasswordHasher` usa PBKDF2-HMAC-SHA256, sale casuale di 16 byte, 210.000 iterazioni e chiave di 256 bit nei parametri predefiniti; confronta l'hash con `MessageDigest.isEqual`. La password non è salvata in chiaro.

Il conto dimostrativo mantiene l'invariante del saldo in `BankAccount.withdraw`: un importo nullo o non positivo è rifiutato; se il saldo è inferiore all'importo il metodo lancia `DomainRuleViolationException`; solo dopo aggiorna il valore in memoria. Le varianti per IBAN di deposito e prelievo in `BankAccountService` recuperano il conto con `findByIbanForUpdate`; le varianti per ID cliente e i flussi di noleggio e vendita usano `findByClientId`, senza quel lock. Questa distinzione è rilevante sotto concorrenza: la presenza di un lock in un percorso non lo estende agli altri.

Nella UI, `RegistrationController` converte il campo password in `char[]` e lo azzera dopo la richiesta. La route successiva dipende dal ruolo restituito. `AuthService.updateEmail`, `updatePassword` e `deleteAccount` richiedono la password corrente; l'eliminazione è rifiutata se il DAO trova operazioni collegate. Tali controlli non eliminano le lacune seguenti, che vanno tenute separate dalla proprietà dell'hash:

- `RegistrationController` espone pubblicamente anche `SALESMAN` e `MANAGER`; è una scelta da prototipo, non provisioning sicuro dello staff.
- `DatabaseConfig` offre una password DB di fallback e seed demo attivo per default, in contrasto con `README.md`. I valori devono essere scelti esplicitamente prima di un'installazione non dimostrativa.
- Il guard `Route.allows` protegge la navigazione, mentre i servizi ricevono ID da parametri e non verificano il ruolo o la proprietà di ogni operazione. `ServiceUiGateway.completePurchase` ignora il `clientId` ricevuto nel controllo del proprietario della `Request`.
- `AbstractController.showError` mostra `Throwable.getMessage()`. Le eccezioni dei DAO o dei servizi potrebbero quindi rivelare dettagli oltre a un messaggio d'uso; manca una traduzione centralizzata degli errori.

3.2.6 Observer e gestione dell'inventario

`SalesService` e `RentalService` implementano `InventorySubject`. Gli observer sono conservati in una `CopyOnWriteArrayList`. `InventoryEvent` trasporta tipo, ID e targa del veicolo, stato precedente/corrente e istante. Il codice produce, fra gli altri, `STATUS_CHANGED`, `REQUEST_EXPIRED`, `TEST_DRIVES_CANCELLED_VEHICLE_UNAVAILABLE` e

RENTALS_CANCELLED_STATE_CHANGE. Il literal STOCK_RECEIVED nell'enum non prova l'esistenza di un modulo stock, assente dai sorgenti correnti.

La notifica agli observer avviene durante la transazione del servizio, prima del commit. L'handler cattura eccezioni dei subscriber e continua. Queste due scelte impediscono all'Observer di far fallire una vendita per errore di un listener, ma un rollback successivo può lasciare un evento già osservato per una modifica non persistita.

3.3 Presentation Layer

Il livello di presentazione collega le viste JavaFX ai casi d'uso del Business Layer. Le schermate sono definite nei file FXML; i controller leggono gli input, invocano UiGateway e aggiornano i controlli grafici con i record di UiModels. Non costruiscono DAO né connessioni JDBC. DriveHub raccoglie le funzioni di ciascun ruolo in un workspace con più schede. Il diagramma delle classi della figura 2.25 mostra i componenti e le loro dipendenze.

3.3.1 Controller e aree applicative

I sei controller corrispondono alle tre pagine pubbliche e ai tre workspace autenticati:

- **WelcomeController**: apre login e registrazione;
- **LoginController**: invia le credenziali al gateway, apre la sessione e passa al workspace del ruolo autenticato;
- **RegistrationController**: raccoglie anagrafica, credenziali e ruolo, registra l'utente e apre la sessione;
- **CustomerWorkspaceController**: gestisce catalogo, richieste di acquisto, saldo finale, noleggi, test drive, proposta di cessione, conto dimostrativo e profilo;
- **SalesmanWorkspaceController**: mostra veicoli e richieste in attesa, approva o rifiuta le proposte, autorizza gli acquisti e cambia la destinazione dei veicoli tra vendita e noleggio;
- **ManagerWorkspaceController**: gestisce marche, modelli, veicoli e sconti e consulta l'elenco delle vendite.

AbstractController, ereditato da tutti salvo WelcomeController, centralizza l'accesso al gateway e alla sessione, la conversione degli input numerici e la visualizzazione di messaggi di successo o errore. I controller restano responsabili anche di alcune verifiche utili all'interazione, come la presenza di una selezione e l'abilitazione delle date nei calendari. Molti controlli di stato vengono ripetuti dai servizi, ma non tutte le verifiche della UI lo sono: il saldo dell'acquisto, per esempio, non ricontrolla nel servizio autorizzazione, scadenza e identità del cliente.

3.3.2 Creazione dei controller e navigazione

DriveHubApplication.start costruisce il gateway, lo inserisce in ApplicationContext e inizializza NavigationManager. Quando viene aperta una route, FXMLLoader crea il controller e inietta i campi @FXML; subito dopo NavigationManager.show gli passa il contesto, se implementa ContextAware. L'ordine è rilevante: initialize() prepara i

controlli, mentre `onContextReady()` può interrogare il gateway e popolare la vista. Nei workspace, quest'ultimo metodo richiama `refresh()` dopo aver ricevuto il contesto.

```
FXMLLoader loader = new FXMLLoader(resource);
Parent root = loader.load();
Object controller = loader.getController();
if (controller instanceof ContextAware aware) {
    aware.setApplicationContext(context);
}
```

Listing 3.6: Passaggio del contesto dopo il caricamento della vista.

La classe `Route` associa a ogni vista il file FXML e l'insieme dei ruoli ammessi. `NavigationManager.show` controlla la sessione prima di caricare una route privata; `showHomeForCurrentRole` apre il workspace di Customer, Salesman o Manager; `logout` svuota la sessione e torna a `WELCOME`. Il controllo della route protegge la navigazione, ma non sostituisce l'autorizzazione delle singole operazioni. In particolare, `ServiceUiGateway.completePurchase` riceve gli identificatori del cliente e della richiesta e delega al servizio senza confrontarli con la sessione: il requisito `RNF-SEC-02` non è quindi garantito universalmente dal solo Presentation Layer.

3.3.3 Gateway, dati della UI e aggiornamento

`UiGateway` è il contratto consumato dai controller; `ServiceUiGateway` lo implementa usando i servizi applicativi. Le query come `searchCatalog`, `clientRentals` e `allSales` restituiscono record di `UiModels`, adatti alle tabelle JavaFX. I comandi come `bookRental`, `bookTestDrive` e `requestPurchase` sono invece `void`: dopo un esito positivo il controller richiama `refresh()` e rilegge i dati. Questo evita di considerare la copia mostrata nella tabella come stato aggiornato del database.

Nel workspace del cliente, ad esempio, `refresh()` usa l'identificatore della sessione per ricaricare richieste, vendite, noleggi e test drive; aggiorna poi il conto tramite `getBankAccount`. I workspace del venditore e del manager ricaricano analogamente le rispettive liste dopo le operazioni. `SessionContext` conserva solo la sessione corrente, mentre saldo, catalogo e pratiche sono letti dal gateway. Un preventivo di noleggio viene mostrato prima della conferma, ma `RentalService.bookRental` ricalcola l'importo quando esegue il caso d'uso: il valore a schermo non è il vincolo di prezzo.

La registrazione espone nella `ComboBox` anche i ruoli Salesman e Manager, oltre a Client. È una scelta presente nell'interfaccia corrente; in un sistema distribuito l'assegnazione dei ruoli privilegiati richiederebbe un percorso amministrato. Anche per questo le guardie di navigazione e i controlli dei controller vanno letti come comportamento della UI, non come prova di autorizzazione end-to-end.

3.4 Persistenza: livello DAO e mapping oggetti/tabelle

Il livello DAO traduce oggetti del dominio e righe SQL senza ricorrere a un ORM. Le query e il mapping sono scritti in JDBC, con `PreparedStatement` per i parametri. Le interfacce del package `dao.interfaces` costituiscono le porte usate dai servizi; le classi `dao.postgres` ne forniscono le implementazioni. Il diagramma della figura 2.24 mostra factory, unità di lavoro e DAO rappresentativi.

3.4.1 Porte DAO e implementazioni JDBC

DriveHub definisce una porta per ciascuna area persistente, come `UserDao`, `VehicleDao`, `RentalDao` e `SaleDao`. Non c'è un'interfaccia CRUD generica: ciascuna porta espone le query necessarie ai propri casi d'uso. Per esempio, `UserDao` offre ricerche per ID, email e ruolo, oltre alla verifica delle operazioni collegate all'account.

```
Optional<User> findById(long id);
Optional<User> findByEmail(String normalizedEmail);
List<User> findByRole(Role role);
List<User> findAll();
boolean hasOperations(long userId, Role role);
```

Listing 3.7: Operazioni di lettura e controllo dichiarate da UserDao.

`PostgresUserDao` implementa questa porta e scrive nella tabella del ruolo. Gli altri `Postgres*Dao` gestiscono catalogo, sconti, conto e operazioni; `JdbcEntityLoader` concentra il caricamento per ID di più entità. `JdbcSupport` raccoglie operazioni JDBC comuni, fra cui inserimento con chiave generata e gestione dei valori interi nullable. Il servizio riceve una `DaoFactory`, quindi non dipende dalle classi JDBC concrete.

3.4.2 Mapping degli utenti e vista unificata

Lo schema contiene tre tabelle, `client`, `salesman` e `manager`. La vista `user` le combina con `UNIONALL` e aggiunge la colonna `role`; non è una quarta tabella. `PostgresUserDao.save` seleziona la tabella mediante il ruolo dell'oggetto, mentre le ricerche comuni leggono dalla vista. Il caricatore costruisce un `User` con i campi condivisi e converte la stringa SQL in `Role`:

```
return load("SELECT id, name, surname, email, "
       + "hash_password, role FROM \"user\" WHERE id = ?", id,
       row -> new User(
              row.getLong("id"),
              row.getString("name"),
              row.getString("surname"),
              row.getString("email"),
              row.getString("hash_password"),
              Role.fromString(row.getString("role"))));
```

Listing 3.8: Ricostruzione di User da una riga della vista user.

La vista non possiede una chiave primaria globale: le tre tabelle generano gli ID indipendentemente. Il metodo `JdbcEntityLoader.user(id)` interroga la vista con il solo ID e prende la prima riga; se due ruoli hanno lo stesso numero, il risultato può essere ambiguo. Le operazioni di modifica di `PostgresUserDao` ricevono invece anche il ruolo per scegliere la tabella. La ricerca per email usa `LOWER(email)=LOWER(?)`; l'unicità SQL dell'email è definita nelle singole tabelle, mentre il percorso di registrazione controlla anche la vista comune. I trigger di disgiunzione previsti da `init.sql` sono installati su PostgreSQL, ma non su H2 nella modalità dimostrativa; su PostgreSQL il confronto degli ID numerici fra tabelle diverse ne limita comunque la garanzia, come spiegato nella sezione 2.8.2.

3.4.3 Mapping delle operazioni e valori opzionali

`JdbcEntityLoader` converte gli identificatori esterni in campi dell'entità e i tipi SQL `DATE` e `TIMESTAMP` in `LocalDate` e `LocalDateTime`. La tabella `sale` consente che `vehicle` e `salesman` siano `NULL`; è necessario conservarli tali nel modello Java, perché una penale può non riferirsi a un veicolo e a un venditore. Il mapping di `Sale` usa quindi `JdbcSupport.nullableLong`:

```
JdbcSupport.nullableLong(row, "vehicle"),
JdbcSupport.nullableLong(row, "salesman"),
```

Listing 3.9: Lettura delle due chiavi opzionali nel mapping di Sale.

In scrittura, `PostgresSaleDao.save` usa `JdbcSupport.setNullableLong` per gli stessi campi. Una lettura diretta con `ResultSet.getLong` restituirebbe invece zero per un valore SQL nullo se non fosse accompagnata da `wasNull()`. Il modello relazionale della sezione 2.8.2 precisa quali associazioni sono obbligatorie e quali opzionali.

3.4.4 Vincoli SQL e controlli dei servizi

Lo schema aggiunge vincoli che operano anche se un dato raggiunge il database senza passare dai controlli della schermata: `UNIQUE` su VIN e targa, `CHECK` su saldo, prezzi, sconti e stati del veicolo, chiavi esterne fra le operazioni e i rispettivi soggetti. Tre esempi dall'inizializzazione SQL sono:

```
CONSTRAINT "uq_vehicle_vin" UNIQUE ("vin"),
CONSTRAINT "chk_bank_account_balance" CHECK ("balance" >= 0),
CONSTRAINT "chk_rent_dates" CHECK ("end" >= "start")
```

Listing 3.10: Vincoli fisici presenti in init.sql.

I vincoli fisici non coprono tutte le regole temporali. Per un noleggio, il metodo `hasOverlappingRentals` di `PostgresRentalDao` usa il predicato del listato 3.5; il servizio verifica il risultato prima dell'inserimento. La query parametrizzata evita di incorporare i valori nella stringa SQL, ma la lettura e l'`INSERT` non sono protetti da un vincolo di esclusione contro richieste concorrenti. Analogamente, lo slot di un test drive viene controllato dal servizio e non da una chiave unica nello schema. `PostgresVehicleDao.updateState` aggiorna per ID senza confrontare nella `WHERE` lo stato precedente. La tabella 3.2 riassume la portata delle garanzie.

3.4.5 Connessione condivisa e Unit of Work

`DaoFactory.begin` apre una `UnitOfWork`. `PostgresDaoFactory` ottiene una connessione dal `DataSource`, disabilita l'auto-commit e imposta `TRANSACTION_READ_COMMITTED`. `PostgresUnitOfWork` costruisce tutti i DAO con quella connessione e lo stesso `JdbcEntityLoader`: i metodi dei DAO non ricevono quindi la connessione a ogni chiamata. Il listato 3.3 mostra come il Business Layer esegue il lavoro, conferma il risultato o chiede il rollback. La chiusura dell'unità annulla una transazione rimasta incompleta.

Il rollback copre le scritture della singola unità prima del commit. Non unifica chiamate applicative separate: richiesta di acquisto, autorizzazione e saldo sono tre transazioni. Non revoca neppure un evento Observer pubblicato durante la lambda, prima del commit. Una

chiamata annidata a `TransactionRunner.execute`, come nel percorso di `SalesService.approveVehicle`, apre un'altra unità di lavoro. Il bootstrap SQL usa separatamente `JdbcTransactionManager`.

Tabella 3.2: Garanzie transazionali e vincoli nel checkout corrente.

Regola	Meccanismo osservato	Limite della garanzia
Scritture di un caso d'uso	Connessione condivisa e commit/rollback di `UnitOfWork`	Non include side effect degli Observer o transazioni annidate
Saldo non negativo	Invariante `BankAccount`, `CHECK` SQL	`findByClientId` non acquisisce lock; una lettura concorrente può essere obsoleta
VIN e targa unici	`UNIQUE` nella tabella `vehicle`	L'errore DB richiede una traduzione per la UI
Test drive nello stesso slot	Query `hasConflict` nel servizio	Manca `UNIQUE(vehicle, slot)`
Noleggi non sovrapposti	Query `hasOverlap`	Manca un vincolo/esclusione concorrente
Transizione di stato veicolo	Verifica nel servizio e `UPDATE` per ID	Manca confronto atomico con stato atteso

3.5 Idiomi e Design Pattern

Le scelte ricorrenti dell'implementazione sono qui ricondotte ai pattern che le descrivono. I paragrafi precedenti ne mostrano il ruolo nei casi d'uso; questa sezione chiarisce quali classi collaborano, quale problema risolvono e dove si arresta la garanzia offerta dal codice corrente.

3.5.1 Observer

`RentalService` e `SalesService` implementano `InventorySubject`: mantengono una lista di `InventoryObserver` e comunicano variazioni dell'inventario con `InventoryEvent`. Un listener riceve il tipo dell'evento, il veicolo e gli stati coinvolti senza che il servizio debba conoscere il comportamento del destinatario. L'interfaccia dell'observer è funzionale:

```
@FunctionalInterface
public interface InventoryObserver {
    void onInventoryEvent(InventoryEvent event);
}
```

Listing 3.11: Contratto dell'osservatore dell'inventario.

I servizi usano `CopyOnWriteArrayList` per conservare gli iscritti e intercettano le eccezioni dei callback. La notifica avviene però nella lambda transazionale prima del

commit: un osservatore può vedere un evento riferito a una modifica poi annullata. Il pattern disaccoppia mittente e ricevente, ma qui non garantisce consegna dopo il commit; il comportamento è dettagliato nel paragrafo 3.2.6.

3.5.2 Strategy

`PricingStrategy` definisce le operazioni di preventivo per acquisto, noleggio, acconto e valutazione del veicolo. `StandardPricingStrategy` contiene le formule correnti; `PricingService` riceve l'interfaccia nel costruttore e le delega i calcoli. Il costruttore usato dalla composizione normale sceglie l'implementazione standard:

```
public PricingService(DaoFactory daoFactory) {
    this(daoFactory, new StandardPricingStrategy());
}

public PricingService(DaoFactory daoFactory, PricingStrategy pricingStrategy) {
    this.transactions = new TransactionRunner(daoFactory);
    this.pricingStrategy = Objects.requireNonNull(pricingStrategy, "
        pricingStrategy");
}
```

Listing 3.12: Strategia di prezzo scelta di default e iniettata nel servizio.

Il servizio può quindi usare un'altra strategia senza incorporarne le formule. Il codice distribuisce una sola strategia concreta e non offre una scelta di algoritmo all'utente. Gli sconti e i diversi importi usati nei flussi sono analizzati nel paragrafo 3.2.4.

3.5.3 Metodi statici di creazione

Le entità espongono metodi come `Vehicle.create`, `Rent.create`, `Request.create` e `User.create`. Questi metodi rendono esplicita la creazione di una nuova entità con ID non ancora assegnato; il costruttore verifica poi le invarianti. Per esempio:

```
public static Vehicle create(long modelId, String vin, String licencePlate,
                             int kilometers, BigDecimal price, int year,
                             VehicleState state) {
    return new Vehicle(null, modelId, vin, licencePlate,
                       kilometers, price, year, state);
}
```

Listing 3.13: Creazione di un veicolo nuovo con ID da assegnare.

Si tratta di metodi factory statici, non di una gerarchia Factory Method con sottoclassi che scelgono il prodotto. Inoltre i costruttori delle entità restano pubblici: `create` è il percorso espressivo usato dai servizi, non l'unico modo consentito dal linguaggio per costruire l'oggetto. `PostgresDaoFactory` svolge una funzione diversa: apre unità di lavoro JDBC che forniscono i DAO.

3.5.4 Istanza centrale per la navigazione

`NavigationManager` conserva un riferimento statico all'istanza corrente. All'avvio, `DriveHubApplication` chiama `initialize` con lo stage e il contesto; i controller usano

`get()` per cambiare route o uscire dalla sessione. In questo senso il gestore è usato come un Singleton applicativo:

```
private static NavigationManager instance;

public static void initialize(Stage stage, ApplicationContext context) {
    instance = new NavigationManager(stage, context);
}
```

Listing 3.14: Inizializzazione del gestore di navigazione.

`initialize` può sostituire l'istanza e non è sincronizzato: il codice assume l'inizializzazione sul thread dell'applicazione JavaFX, non implementa un Singleton immutabile e sicuro per accessi concorrenti. Il gestore non è invece un catalogo globale di servizi. Dopo che `FXMLLoader` ha creato il controller, `NavigationManager.show` gli passa `ApplicationContext` tramite `ContextAware`; il controller usa quel contesto per ottenere `UiGateway`, come mostrato nel paragrafo 3.3.

3.5.5 DAO e Unit of Work

Il pattern DAO separa le operazioni persistenti dai servizi: `UserDao` e `VehicleDao` sono porte, mentre `PostgresUserDao` e `PostgresVehicleDao` traducono oggetti e query JDBC. La distinzione rende possibile sostituire l'adapter attraverso `DaoFactory`, senza far conoscere SQL a controller e servizi. Il mapping concreto, inclusa la vista `user`, è descritto nel paragrafo 3.4.

Unit of Work raggruppa i DAO necessari a un caso d'uso su una sola connessione. `PostgresDaoFactory.begin` apre la transazione; `PostgresUnitOfWork` costruisce i DAO che la condividono; `TransactionRunner.execute` esegue la lambda e decide commit o rollback. Il listato 3.3 mostra il confine applicativo. L'unità è limitata a una chiamata di `execute`: non comprende le operazioni successive né una seconda unità aperta da una chiamata annidata. I limiti sono trattati nel paragrafo 3.4.5.

3.5.6 Idiomi Java usati nei confini tra livelli

Le query DAO restituiscono spesso `Optional<T>`: l'assenza di una riga è distinta da un'entità presente e il servizio la trasforma, quando necessario, in `EntityNotFoundException`. I record di `UiModels` sono contenitori di dati per la UI, mentre `RegistrationRequest` trasporta i campi della registrazione; non sostituiscono le entità modificabili del dominio. Le lambda `Function<UnitOfWork,T>` permettono ai servizi di esprimere il lavoro transazionale senza gestire direttamente commit e rollback. Infine il `try`-with-resources di `TransactionRunner` chiude l'unità di lavoro anche in caso di eccezione. Sono idiomi della piattaforma Java che sostengono i confini dell'architettura, senza introdurre ulteriori livelli applicativi.

Capitolo 4

Testing

4.1 Strategia di verifica

La verifica principale è organizzata su due livelli, come i confini dell'architettura. Il primo controlla regole del dominio, calcoli e servizi, isolando con Mockito i DAO quando l'obiettivo è osservare una decisione del Business Layer. Il secondo esegue le implementazioni JDBC su un database H2 in memoria con `MODE=PostgreSQL`, così da verificare query, mapping e vincoli invece delle sole chiamate fra mock.

- **Livello 1 – Business Logic**: test delle entità, di `PricingStrategy`, dei servizi e di `TransactionRunner`. Gli oracoli sono stati finali, importi, eccezioni e interazioni con le porte DAO.
- **Livello 2 – Persistence Layer**: test di `DatabaseBootstrap` e dei DAO `Postgres*Dao` con tabelle reali in H2. Gli oracoli sono righe salvate e rilette, conteggi dello schema e dati ricostruiti.

DriveHub dispone anche di test funzionali che attraversano servizi e persistenza, di controlli statici dell'architettura e delle viste FXML, e di uno smoke test JavaFX separato. Queste prove sono discusse dopo i due livelli principali, perché hanno un perimetro diverso. Un test con mock può dimostrare che il servizio richiede un aggiornamento e un commit; un test su H2 può dimostrare che la query funziona in quell'ambiente. Nessuno dei due dimostra, da solo, il comportamento di due transazioni concorrenti su PostgreSQL.

4.1.1 Tecnologie e strumenti

- **JUnit 5** esegue test e assert, con `@BeforeEach` per preparare un contesto nuovo prima di ogni prova. `@ParameterizedTest` verifica i tre ruoli nella registrazione e nel login.
- **Mockito** sostituisce `DaoFactory`, `UnitOfWork` e singoli DAO nei test dei servizi. `verify` controlla le chiamate; `inOrder` ne controlla la sequenza nel test del rollback.
- **H2 in modalità PostgreSQL** ospita i test DAO e i flussi funzionali. Lo schema è caricato da `DatabaseBootstrap`; ogni contesto usa un database in memoria con nome distinto.
- **Maven Surefire** esegue la suite standard. `pom.xml` esclude per default i tag `gui` e `postgres`; JaCoCo è configurato, ma la presenza del plugin non equivale a un report di copertura.

4.2 Livello 1: Business Logic

4.2.1 Invarianti del dominio

`UserAndVehicleTest`, `SalesWorkflowTest` e `RentalWorkflowTest` usano oggetti reali, senza database. Verificano, fra l'altro, normalizzazione dell'utente, transizioni del veicolo, targa assente con chilometri zero, creazione della richiesta e validità di noleggio e test drive. Abbiamo scelto questi casi perché una vista può impedire un input errato, ma un servizio o un altro adapter possono costruire le entità direttamente. La regola deve quindi rimanere verificabile nel dominio.

4.2.2 AuthServiceTest

La registrazione parametrizzata prova i quattro valori dell'enum `Role`: `CLIENT`, l'alias storico `CUSTOMER`, `SALESMAN` e `MANAGER`. Il test controlla l'entità restituita, il commit e la creazione del conto per i due valori cliente. Il caso di email duplicata verifica invece che il servizio interrompa il percorso prima di calcolare l'hash e salvare un utente. Gli assert sui mock fissano la decisione osservata, mentre il test funzionale del login verifica separatamente il risultato quando servizi e DAO lavorano insieme.

```
verify(hasher, never()).hash(any());
verify(users, never()).save(any());
verify(unit).rollback();
```

Listing 4.1: Effetti esclusi dal test di registrazione duplicata.

4.2.3 PricingAndSecurityTest

Il test `discountAppliesToSalePrice` controlla due punti della strategia: per un modello da 40.000 euro, veicolo dell'anno corrente senza targa e a zero chilometri, il preventivo è 36.000 euro con promozione del 10% e 40.000 euro senza promozione. `discountAppliesToRental` verifica gli importi per uno, due, tre e quattro giorni. Sono assert sul calcolo, indipendenti dalla persistenza:

```
assertEquals(new BigDecimal("40.00"), strategy.rentalPrice(target, 1));
assertEquals(new BigDecimal("76.00"), strategy.rentalPrice(target, 2));
assertEquals(new BigDecimal("108.30"), strategy.rentalPrice(target, 3));
assertEquals(new BigDecimal("137.18"), strategy.rentalPrice(target, 4));
```

Listing 4.2: Estratto degli assert sulla tariffa di noleggio.

Nella stessa classe, `pbkdf2RoundTrip` controlla che due hash della medesima password abbiano sali diversi, che la password corretta sia accettata e che una password errata o una codifica malformata siano rifiutate. Il test usa parametri PBKDF2 ridotti per l'esecuzione, quindi verifica il comportamento dell'implementazione senza misurare la resistenza dei parametri di produzione.

4.2.4 SalesServiceTest

Il test di avvio acquisto prepara veicolo e conto mediante mock. Dopo la richiesta controlla tre effetti collegati: l'acconto diminuisce il saldo, il veicolo passa a `BLOCKED` e la richiesta

viene salvata; verifica poi il commit. Il test di annullamento copre il ramo alternativo: il veicolo torna `SALE`, il cliente riceve l'80% dell'acconto e una `Sale` senza veicolo e venditore registra il 20% trattenuto.

```
verify(vehicles).updateState(1L, VehicleState.SALE);
assertEquals(new BigDecimal("5800.00"), account.balance());
verify(requests).delete(50L);
verify(sales).save(argThat(sale ->
       sale.vehicleId() == null && sale.salesmanId() == null &&
       sale.amount().compareTo(new BigDecimal("200.00")) == 0));
```

Listing 4.3: Oracoli del test di annullamento con penale.

Questi test accertano la decisione del servizio e le chiamate ai DAO simulati. Non sono una prova che le scritture arrivino su PostgreSQL: quel confine è affrontato dai test di persistenza e dai test funzionali.

4.2.5 RentalServiceTest e BankAccountServiceTest

`RentalServiceTest.bookRentalDebitsAccount` prenota un giorno di noleggio per un veicolo con tariffa di 100 euro e saldo iniziale di 500 euro. Il test verifica `Rent`, saldo di 400 euro, aggiornamento del conto, salvataggio del noleggio e commit. `BankAccountServiceTest` aggiunge i casi di deposito, prelievo e saldo insufficiente; quest'ultimo verifica il rifiuto prima di attribuire un saldo negativo al conto.

```
Rent rent = service.bookRental(1L, 30L, start, end);
assertNotNull(rent);
assertEquals(new BigDecimal("400.00"), account.balance());
verify(bankAccounts).updateBalance(account.iban(), account.balance());
verify(rentals).save(any(Rent.class));
verify(unit).commit();
```

Listing 4.4: Effetti economici verificati nel test del noleggio.

4.2.6 TransactionAndObserverTest

`commitFailurePreservesCauseAndCloses` programma un errore di commit e uno successivo di rollback. L'oracolo richiede che il chiamante riceva l'errore originale, che il secondo sia aggiunto come eccezione soppressa e che `UnitOfWork.close` venga invocato dopo commit e rollback:

```
assertArrayEquals(new Throwable[]{rollback}, original.getSuppressed());
var order = inOrder(unit);
order.verify(unit).commit();
order.verify(unit).rollback();
order.verify(unit).close();
```

Listing 4.5: Ordine delle operazioni nel test dell'errore transazionale.

Gli altri test della classe osservano l'evento emesso da `RentalService` quando cancella test drive incompatibili e l'evento di `SalesService` quando blocca un veicolo. Verificano l'emissione nell'esecuzione corrente, non una consegna dopo il commit o un recupero degli eventi dopo un riavvio, poiché l'observer è locale al processo.

4.3 Livello 2: Persistence Layer (DAO)

Questi test eseguono query JDBC su tabelle create dalla migration iniziale. Un nuovo database H2 viene preparato prima di ogni prova; le query non sono simulate. La modalità di compatibilità è utile per controllare SQL e mapping, ma non sostituisce PostgreSQL reale per dialetto, trigger e concorrenza.

4.3.1 PostgresDaoIntegrationTest

Nonostante il nome della classe, il database impiegato è H2. Il primo test salva i tre ruoli, catalogo, conto e sconti, poi verifica i dati riletti. Il secondo salva un noleggio e un test drive, conferma la transazione, li rilegge in una nuova `UnitOfWork`, salva una vendita e la rilegge ancora. Il passaggio a una nuova unità è decisivo: impedisce che l'assert osservi soltanto gli oggetti Java rimasti in memoria.

```
try (var work = daoFactory.begin()) {
    assertEquals(1, work.rentals().findByClientId(clientId).size());
    assertEquals(1, work.testDrives().findByClientId(clientId).size());
    work.commit();
}
```

Listing 4.6: Verifica di lettura in una nuova Unit of Work.

Il test controlla il percorso nominale di salvataggio e ricostruzione. Non mette due transazioni in competizione per lo stesso veicolo né verifica la semantica dei trigger su PostgreSQL.

4.3.2 DatabaseBootstrapIntegrationTest

Il primo test controlla una voce nel registro delle migration e le tredici tabelle applicative. Il secondo esegue due volte il bootstrap con seed e verifica che il catalogo sia popolato. Il terzo applica `db/demo.sql` e confronta i conteggi delle entità, incluse otto vendite con veicolo e venditore nulli. Questi assert sono utili perché un errore nello schema o nel dataset di prova renderebbe poco affidabili anche i test dei servizi che vi si appoggiano.

```
assertEquals(56, scalarLong(connection,
       "SELECT COUNT(*) FROM \"vehicle\""));
assertEquals(40, scalarLong(connection,
       "SELECT COUNT(*) FROM \"sale\""));
assertEquals(8, scalarLong(connection,
       "SELECT COUNT(*) FROM \"sale\" WHERE \"vehicle\" IS NULL AND \"salesman\"
           IS NULL"));
```

Listing 4.7: Conteggi controllati dopo il caricamento dei dati dimostrativi.

4.4 Verifiche funzionali e della presentazione

`FunctionalTestSupport` costruisce servizi e `ServiceUiGateway` reali sopra i DAO JDBC e un database H2 isolato. `CheckoutFunctionalTest.completePurchaseWorkflow` percorre acconto, autorizzazione e saldo, osservando `BLOCKED`, poi `SOLD`, eliminazione della richiesta, vendita e saldo finale. `cancelPurchaseWorkflowWithPenalty` verifica rimborso

parziale e vendita di penale; gli altri due metodi coprono noleggio e proposta di cessione. Sono prove che attraversano più livelli, ma esercitano i percorsi scelti e non tutte le varianti o gli accessi concorrenti.

```
Vehicle blocked = transactions.execute(u ->
       u.vehicles().findById(saleVehicle.requireId()).orElseThrow());
assertEquals(VehicleState.BLOCKED, blocked.state());

Vehicle sold = transactions.execute(u ->
       u.vehicles().findById(saleVehicle.requireId()).orElseThrow());
assertEquals(VehicleState.SOLD, sold.state());
```

Listing 4.8: Due stati osservati nel test funzionale di acquisto.

`UserGoalsFunctionalTest` verifica registrazione e login dei tre ruoli, email duplicata, credenziali errate, catalogo, test drive e applicazione/rimozione di uno sconto. `LayeringTest` controlla staticamente la direzione delle dipendenze fra package; `FxmlContractTest` analizza sei viste, campi `fx:id`, handler e CSS. Questi ultimi test verificano struttura dei sorgenti e contratto FXML, non l'interazione visiva completa. Lo smoke `JavaFxSmokeTest`, marcato `@Tag("gui")`, carica route e verifica guardia e logout in un ambiente grafico; è escluso dalla suite standard descritta sotto.

4.5 Esecuzione della suite e portata dei risultati

Il 21 ottobre 2026 è stata eseguita la suite Maven standard con JDK 21:

```
mvn -B -ntp -Dmaven.repo.local=C:/DEV/.m2/repository test
```

Il risultato è stato **57 test, 0 failure, 0 error, 0 skipped; BUILD SUCCESS**. Il riepilogo Surefire comprende 16 classi. Il profilo standard esclude `gui` e `postgres`; il risultato non include quindi lo smoke JavaFX né prove su un server PostgreSQL. JaCoCo ha segnalato l'assenza del file dei dati di esecuzione e non ha prodotto una percentuale di coverage per questa esecuzione. I risultati full-stack e la copertura in `docs/evidence/full-stack/` appartengono a un manifest precedente e non sono sommati a questa suite.

La suite attesta i casi e gli ambienti appena descritti. Non fornisce una prova di serializzazione delle prenotazioni concorrenti, di unicità globale degli ID nella vista `user`, né di tutte le regole di autorizzazione su chiamate dirette ai servizi. Tali limiti sono coerenti con l'analisi dei vincoli JDBC nel paragrafo 3.4.4; non vanno confusi con un esito negativo dei 57 test eseguiti.